

HUMBER ENGINEERING

MENG-3020

SYSTEMS MODELING & SIMULATION

LECTURE 10

LECTURE 10

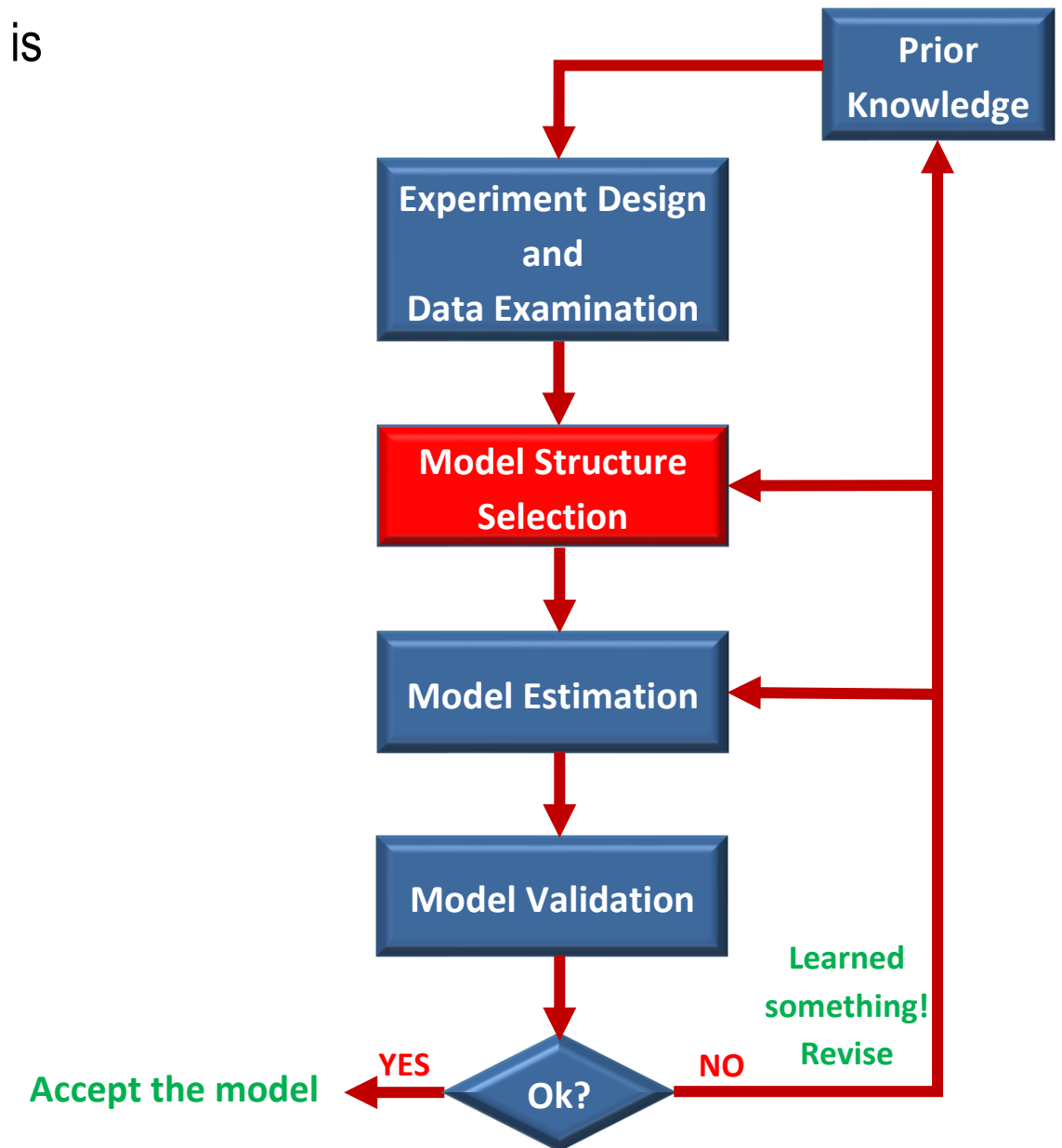
Model Structures & Estimation Methods

- System Identification Procedure
- Linear Regression Models & Least-Squares Estimation
 - Static Systems
 - Dynamic Systems
 - Discrete-time Transfer Function Models
- Determining Model Order and Delay
- Simulation Examples

System Identification Procedure

□ Model Structure Selection

- The **goal** is to determine a **parametric model** for the system. The first step is selecting an **appropriate model structure**.
- **Continuous Time Models**
 - Transfer Function Model
- **Discrete Time Models**
 - Linear Regression Models
 - Transfer Function Models
 - ARX Model
 - Output-Error (OE) Model
 - Box-Jenkins (BJ) Model



What We Already Know?

- The **linear regression models** can be used to model the **static systems** and **dynamic systems**.

$$y(k) = \boldsymbol{\varphi}^T(k) \boldsymbol{\theta} + e(k), \quad k = 1, \dots, N$$

- In modeling of **static systems**, the **regressors** depends only on the **current input data**.
For example:

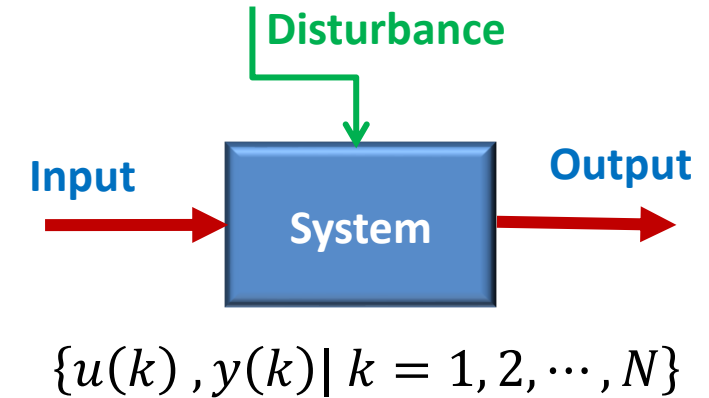
$$y(k) = \theta_0 + \theta_1 u(k) + \theta_2 u(k)^2 + \theta_3 u(k)^3 + e(k)$$

$$y(k) = \underbrace{[1 \quad u(k) \quad u(k)^2 \quad u(k)^3]}_{\boldsymbol{\varphi}^T(k)} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}}_{\boldsymbol{\theta}} + e(k)$$

- In **dynamic systems** modeling, the **regressors** may depend on the **past output** data, and **current and past input** data.
For example:

$$y(k) = \theta_0 y(k-2) + \theta_1 y(k-1) + \theta_2 u(k)^2 + \theta_3 u(k-3) + e(k)$$

$$y(k) = \underbrace{[y(k-2) \quad y(k-1) \quad u(k)^2 \quad u(k-3)]}_{\boldsymbol{\varphi}^T(k)} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}}_{\boldsymbol{\theta}} + e(k)$$



What We Already Know?

□ Least-Squares Estimation

- Assume the general matrix-vector form of the Linear Regression Model
- The **Normal Equation** and **Least-Squares Estimation** are determined as

$$\text{Normal Equation} \rightarrow \Phi^T \mathbf{Y} = \Phi^T \Phi \boldsymbol{\theta}$$

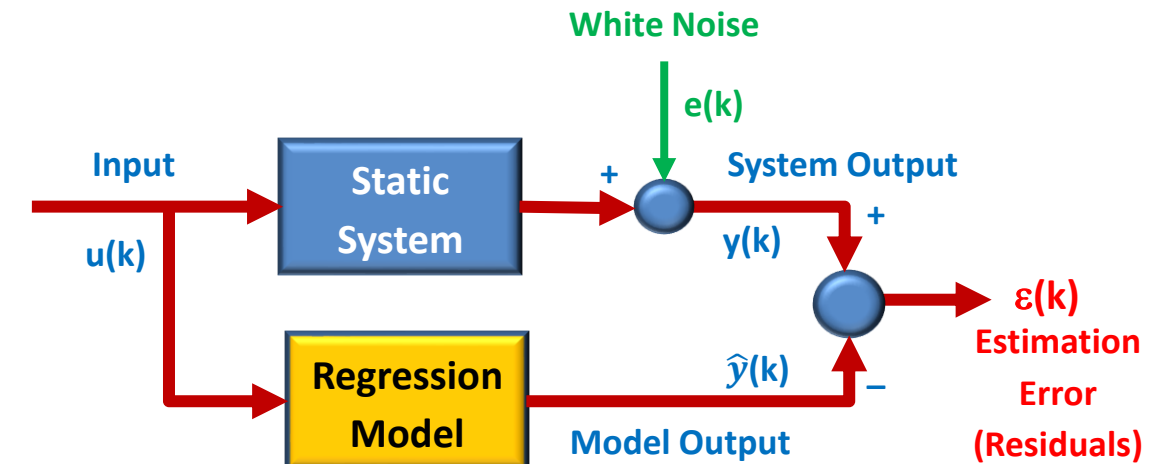
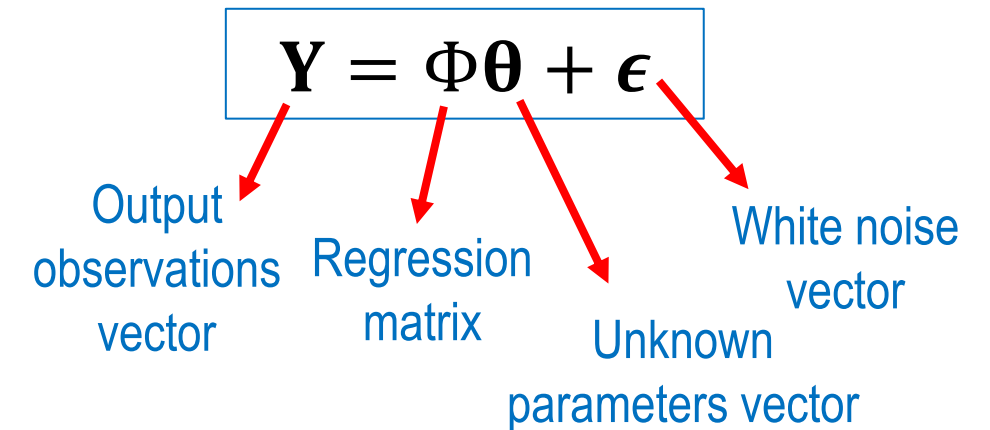
$$\text{Estimated Parameters} \rightarrow \hat{\boldsymbol{\theta}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{Y} = \Phi^\# \mathbf{Y}$$

- The **Least-squares estimation error (residuals)** is defined as the difference between the **observed output data** points and the **estimated output** by the regression model.

$$\text{Least – Squares Estimation Error} \rightarrow \boldsymbol{\varepsilon} = \mathbf{Y} - \hat{\mathbf{Y}} = \mathbf{Y} - \Phi \hat{\boldsymbol{\theta}}$$

- The goal is to estimate the **unknown parameters, $\boldsymbol{\theta}$** , that minimizes the **Mean Square of the Estimation Error**.

$$\text{MSE Loss function} \rightarrow J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{k=1}^N \varepsilon^2(k; \boldsymbol{\theta}) = \frac{1}{N} \|\boldsymbol{\varepsilon}\|_2^2$$



Discrete-time Transfer Function Models

- Consider a general form of a **linear regression model** with additive **white noise**,

$$y(k) + a_1y(k-1) + a_2y(k-2) + \cdots + a_{n_a}y(k-n_a) = b_1u(k-1) + b_2u(k-2) + \cdots + b_{n_b}u(k-n_b) + e(k)$$

- The **backward-shift operator** is defined as

$$q^{-1}x(k) = x(k-1)$$

- We can rewrite the linear regression model in a **polynomial** form:

$$y(k) + a_1q^{-1}y(k) + a_2q^{-2}y(k) + \cdots + a_{n_a}q^{-n_a}y(k) = b_1q^{-1}u(k) + b_2q^{-2}u(k) + \cdots + b_{n_b}q^{-n_b}u(k) + e(k)$$

$$\underbrace{(1 + a_1q^{-1} + a_2q^{-2} + \cdots + a_{n_a}q^{-n_a})}_{A(q)}y(k) = \underbrace{(b_1q^{-1} + b_2q^{-2} + \cdots + b_{n_b}q^{-n_b})}_{B(q)}u(k) + e(k)$$

$$A(q)y(k) = B(q)u(k) + e(k)$$



$$y(k) = \frac{B(q)}{A(q)}u(k) + \frac{1}{A(q)}e(k)$$

- ARX: Auto-Regressive with Exogenous Input**

ARX Model Structure

- Auto-Regressive** ($y(k)$ is a function of previous y values)
- Exogenous Input** ($y(k)$ depends on extra input u)

Discrete-time Transfer Function Models

□ ARX Model (Auto-Regressive with Exogenous Input)

System model

$$G(q) = \frac{B(q)}{A(q)} = \frac{b_1 q^{-1} + b_2 q^{-2} + \dots + b_{n_b} q^{-n_b}}{1 + a_1 q^{-1} + a_2 q^{-2} + \dots + a_{n_a} q^{-n_a}}$$

Noise filter

$$H(q) = \frac{1}{A(q)} = \frac{1}{1 + a_1 q^{-1} + a_2 q^{-2} + \dots + a_{n_a} q^{-n_a}}$$

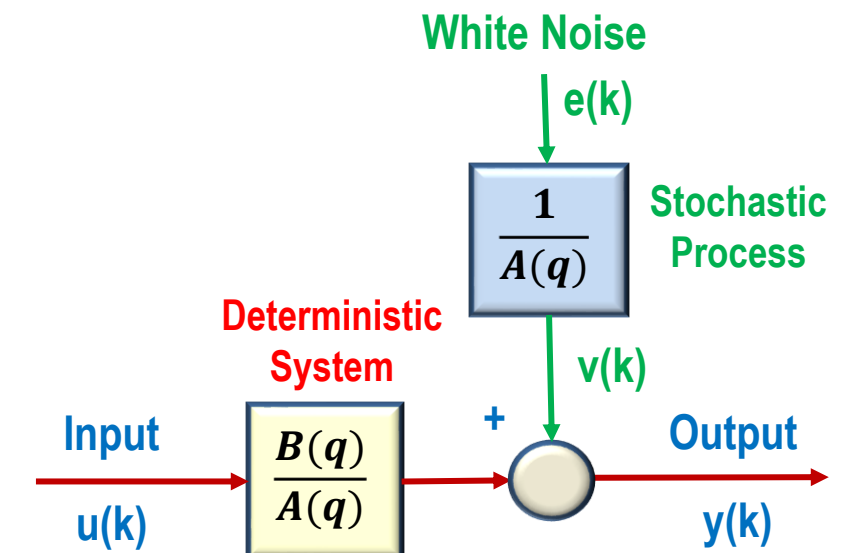
$$y(k) = \frac{B(q)}{A(q)} u(k) + \frac{1}{A(q)} e(k)$$

ARX Model Structure

$$A(q) = 1 + a_1 q^{-1} + a_2 q^{-2} + \dots + a_{n_a} q^{-n_a}$$

$$B(q) = b_1 q^{-1} + b_2 q^{-2} + \dots + b_{n_b} q^{-n_b}$$

- The **ARX model** is the most widely applied linear dynamic model.
- The parameters can be estimated using **linear least-squares** technique, which makes it attractive as an **initial solution** to model identification in practice.
- The ARX model assumes that the noise is **process noise**, which means the **input signal** and the **noise signal** are filtered by **same dynamic**.
- If the process noise **does not meet** the noise assumption made by ARX model, the parameters are estimated **biased** and **non-consistent**.



Discrete-time Transfer Function Models

□ Linear Regression Model of ARX Model Structure

- The difference equation is obtained as below

$$A(q)y(k) = B(q)u(k) + e(k)$$

$$(1 + a_1q^{-1} + \dots + a_{n_a}q^{-n_a})y(k) = (b_1q^{-1} + \dots + b_{n_b}q^{-n_b})u(k) + e(k)$$

$$y(k) + a_1y(k-1) + \dots + a_{n_a}y(k-n_a) = b_1u(k-1) + \dots + b_{n_b}u(k-n_b) + e(k)$$

$$y(k) = -a_1y(k-1) - \dots - a_{n_a}y(k-n_a) + b_1u(k-1) + \dots + b_{n_b}u(k-n_b) + e(k)$$

$$y(k) = \underbrace{[-y(k-1) \quad \dots \quad -y(k-n_a) \quad u(k-1) \quad \dots \quad u(k-n_b)]}_{\boldsymbol{\varphi}^T(k)} \underbrace{\begin{bmatrix} a_1 \\ \vdots \\ a_{n_a} \\ b_1 \\ \vdots \\ b_{n_b} \end{bmatrix}}_{\boldsymbol{\theta}} + \underbrace{e(k)}_{\text{White Noise}}$$

Past input-output data

Unknown parameters



$$y(k) = \boldsymbol{\varphi}^T(k)\boldsymbol{\theta} + e(k)$$

Linear Regression Model

$$y(k) = \frac{B(q)}{A(q)}u(k) + \frac{1}{A(q)}e(k)$$

ARX Model Structure

$$A(q) = 1 + a_1q^{-1} + a_2q^{-2} + \dots + a_{n_a}q^{-n_a}$$

$$B(q) = b_1q^{-1} + b_2q^{-2} + \dots + b_{n_b}q^{-n_b}$$

- Therefore, the ARX model parameters can be estimated by LS method.

Discrete-time Transfer Function Models

Example 1

Consider a dynamic system with sampled input $u(k)$, output $y(k)$ and $e(k)$ is white noise. According to the identification experiments we assume the following model for this system.

The a and b are unknown parameters.

$$y(k) = \frac{bq^{-1}}{1 + aq^{-1}}u(k) + \frac{1}{1 + aq^{-1}}e(k)$$

a) Determine the structure of the model.

Here, the model has an **ARX structure** with the following polynomials:

$$y(k) = \frac{B(q)}{A(q)}u(k) + \frac{1}{A(q)}e(k) \quad \Rightarrow \quad \begin{aligned} A(q) &= (1 + aq^{-1}) \\ B(q) &= bq^{-1} \end{aligned}$$

b) Determine the linear regression model of the given structure.

$$(1 + aq^{-1})y(k) = bq^{-1}u(k) + e(k) \quad \longrightarrow \quad y(k) = -ay(k-1) + bu(k-1) + e(k)$$

$$y(k) = \underbrace{[-y(k-1) \quad u(k-1)]}_{\boldsymbol{\varphi}^T(k)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{\boldsymbol{\theta}} + e(k)$$

Linear Regression Model

Discrete-time Transfer Function Models

Example 1

Consider a dynamic system with sampled input $u(k)$, output $y(k)$ and $e(k)$ is white noise. According to the identification experiments we assume the following model for this system.

The a and b are unknown parameters.

$$y(k) = \frac{bq^{-1}}{1 + aq^{-1}}u(k) + \frac{1}{1 + aq^{-1}}e(k)$$

c) Assume that the following I/O data samples are available. Determine the Least-Squares Estimation of the unknown parameters.

First, generate the **matrix-vector** form of the **linear regression model** based on the given data:

$$y(k) = -ay(k-1) + bu(k-1) + e(k)$$

$$\begin{cases} y(2) = -ay(1) + bu(1) + e(2) \\ y(3) = -ay(2) + bu(2) + e(3) \\ y(4) = -ay(3) + bu(3) + e(4) \end{cases} \quad \xrightarrow{\text{Vector-matrix form}} \quad \underbrace{\begin{bmatrix} y(2) \\ y(3) \\ y(4) \end{bmatrix}}_Y = \underbrace{\begin{bmatrix} -y(1) & u(1) \\ -y(2) & u(2) \\ -y(3) & u(3) \end{bmatrix}}_{\Phi} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{\theta} + \underbrace{\begin{bmatrix} e(2) \\ e(3) \\ e(4) \end{bmatrix}}_{\epsilon}$$

k	1	2	3	4
u	2	3	2	3
y	-23	48	-93	36

The **least squares estimation** of unknown parameters a and b are obtained as below

$$\boxed{\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T Y} \longrightarrow \begin{bmatrix} \hat{a} \\ \hat{b} \end{bmatrix} = (\Phi^T \Phi)^{-1} \Phi^T Y$$

Discrete-time Transfer Function Models

Example 1

Consider a dynamic system with sampled input $u(k)$, output $y(k)$ and $e(k)$ is white noise. According to the identification experiments we assume the following model for this system.

The a and b are unknown parameters.

$$y(k) = \frac{bq^{-1}}{1 + aq^{-1}}u(k) + \frac{1}{1 + aq^{-1}}e(k)$$

c) Assume that the following I/O data samples are available. Determine the Least-Squares estimation of the unknown parameters and from the ARX model.

The least squares estimation of unknown parameters a and b are obtained as below

$$\begin{bmatrix} \hat{a} \\ \hat{b} \end{bmatrix} = (\Phi^T \Phi)^{-1} \Phi^T Y$$

$$\begin{bmatrix} \hat{a} \\ \hat{b} \end{bmatrix} = \left(\begin{bmatrix} -y(1) & -y(2) & -y(3) \\ u(1) & u(2) & u(3) \end{bmatrix} \begin{bmatrix} -y(1) & u(1) \\ -y(2) & u(2) \\ -y(3) & u(3) \end{bmatrix} \right)^{-1} \begin{bmatrix} -y(1) & -y(2) & -y(3) \\ u(1) & u(2) & u(3) \end{bmatrix} \begin{bmatrix} y(2) \\ y(3) \\ y(4) \end{bmatrix}$$

$$\begin{bmatrix} \hat{a} \\ \hat{b} \end{bmatrix} = \left(\begin{bmatrix} 23 & -48 & 93 \\ 2 & 3 & 2 \end{bmatrix} \begin{bmatrix} 23 & 2 \\ -48 & 3 \\ 93 & 2 \end{bmatrix} \right)^{-1} \begin{bmatrix} 23 & -48 & 93 \\ 2 & 3 & 2 \end{bmatrix} \begin{bmatrix} 48 \\ -93 \\ 36 \end{bmatrix} \rightarrow \begin{bmatrix} \hat{a} \\ \hat{b} \end{bmatrix} = \left(\begin{bmatrix} 11482 & 88 \\ 88 & 17 \end{bmatrix} \right)^{-1} \begin{bmatrix} 8916 \\ -111 \end{bmatrix} \rightarrow \begin{bmatrix} \hat{a} \\ \hat{b} \end{bmatrix} = \begin{bmatrix} 0.8607 \\ -10.9848 \end{bmatrix}$$

k	1	2	3	4
u	2	3	2	3
y	-23	48	-93	36

From the estimated parameters $\hat{a} = 0.8607$ and $\hat{b} = -10.9848$ we have the following ARX model

Estimated ARX Model

$$y(k) = \frac{-10.9848q^{-1}}{1 + 0.8607q^{-1}}u(k) + \frac{1}{1 + 0.8607q^{-1}}e(k)$$

Discrete-time Transfer Function Models

❑ Output Error (OE) Model Estimation

- The **OE Model** is a **very simple model** and more realistic process description.
- OE model assumes the disturbance is a **White noise**.

$$y(k) = \frac{B(q)}{F(q)} u(k) + e(k)$$

OE Model Structure

System model

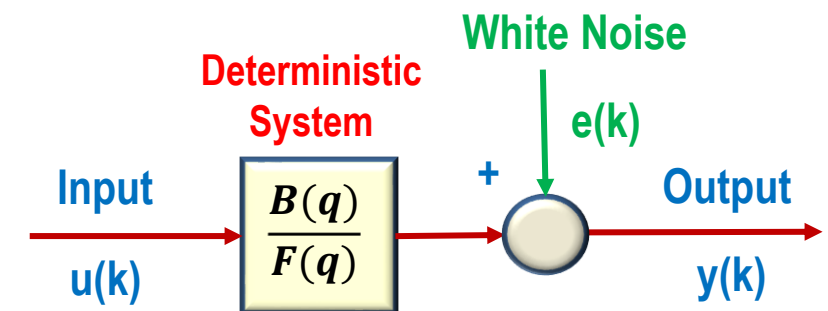
$$G(q) = \frac{B(q)}{F(q)} = \frac{b_1 q^{-1} + b_2 q^{-2} + \dots + b_{n_b} q^{-n_b}}{1 + f_1 q^{-1} + f_2 q^{-2} + \dots + f_{n_f} q^{-n_f}}$$

- Since OE model has **fewer parameters to estimate**, it is often a good option of model structures in **practice**.
- OE model structure provides **good results** when system dominated **additive measurement (sensor) white noise**.
- OE model the parameters **cannot** be estimated by a **simple LS method**.

$$F(q)y(k) = B(q)u(k) + F(q)e(k)$$

$$y(k) + f_1 y(k-1) + \dots + f_{n_f} y(k-n_f) = b_1 u(k-1) + \dots + b_{n_b} u(k-n_b) + e(k) + f_1 e(k-1) + \dots + f_{n_f} e(k-n_f)$$

- The regressors include the **past samples of noise signal** which are **unknown**.
- The parameters of OE model can be estimated by following methods exploiting the relationship to ARX model,
 - Nonlinear Optimization
 - Repeated Least-Squares and Filtering



Discrete-time Transfer Function Models

□ Output Error (OE) Model Estimation

■ Repeated LS and Filtering for OE Model Estimation

- 1) Estimate an ARX model using LS method from the data $\{u(k), y(k)\}$

$$y(k) = \frac{B(q)}{F(q)} u(k) + \frac{1}{F(q)} e(k) \quad \rightarrow \quad F(q)y(k) = B(q)u(k) + e(k)$$

$$y(k) + f_1 y(k-1) + \dots + f_{n_f} y(k-n_f) = b_1 u(k-1) + \dots + b_{n_b} u(k-n_b) + e(k)$$

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{Y}$$

The parameters in $\hat{\theta}$ includes f_i and b_i values

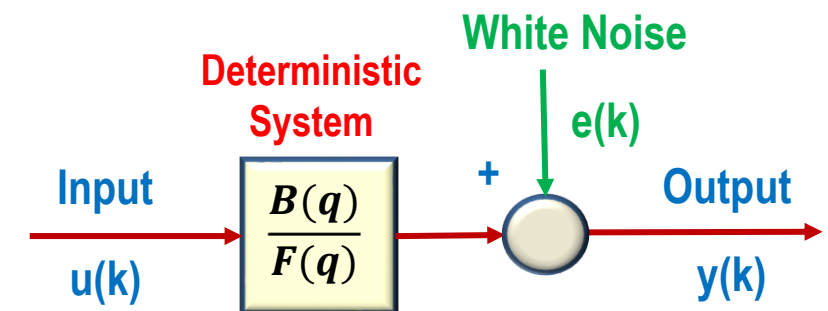
- 2) Filter the input-output data $\{u(k), y(k)\}$ through the estimated filter $\hat{F}(q)$.

$$u^F(k) = \frac{1}{\hat{F}(q)} u(k) \quad y^F(k) = \frac{1}{\hat{F}(q)} y(k)$$

- 3) Estimate the OE model parameters f_i and b_i by an ARX model estimation with the filtered input and output.
- 4) Iterate Steps 2 to 3 until convergence is reached.

$$y(k) = \frac{B(q)}{F(q)} u(k) + e(k)$$

OE Model Structure



Discrete-time Transfer Function Models

□ Box-Jenkins (BJ) Model Structure

- **Box-Jenkins (BJ) Model** is a **very general model**.
- Includes all other models as special case.
- In BJ model, **disturbance** can have a **completely different** model from the **process model**.

$$G(q) = \frac{B(q)}{F(q)} = \frac{b_1 q^{-1} + b_2 q^{-2} + \dots + b_{n_b} q^{-n_b}}{1 + f_1 q^{-1} + f_2 q^{-2} + \dots + f_{n_f} q^{-n_f}}$$

System model

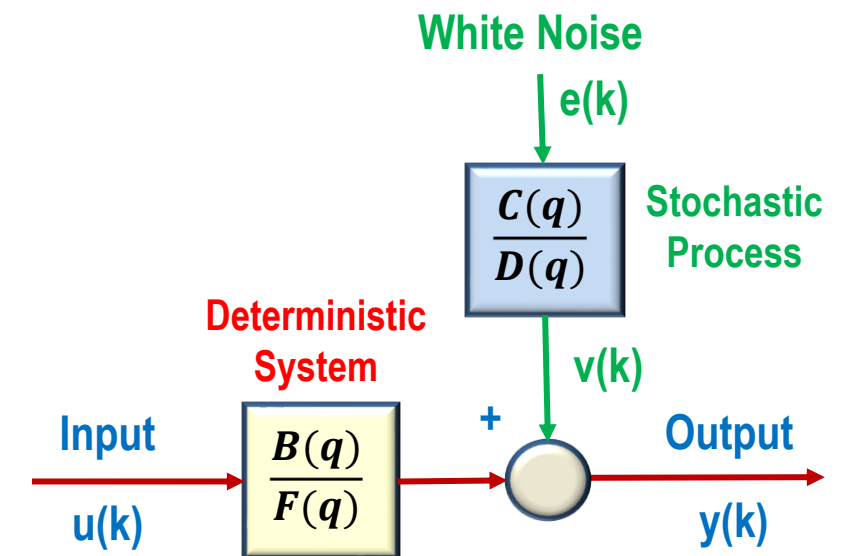
$$H(q) = \frac{C(q)}{D(q)} = \frac{1 + c_1 q^{-1} + c_2 q^{-2} + \dots + c_{n_c} q^{-n_c}}{1 + d_1 q^{-1} + d_2 q^{-2} + \dots + d_{n_d} q^{-n_d}}$$

Noise filter

- Typically, a BJ model can be estimated by following approaches
 - **Nonlinear Optimization**
 - **Independent Parameterization of $G(q)$ and $H(q)$**

$$y(k) = \frac{B(q)}{F(q)} u(k) + \frac{C(q)}{D(q)} e(k)$$

BJ Model Structure



Discrete-time TF Model Estimation in MATLAB

- In [System Identification Toolbox](#) MATLAB, we have the following functions to obtain DT transfer function model structures

- The **arx** function estimates parameters of [ARX model](#)

```
sys = arx(data,[na nb nk])
```

- The **oe** function estimates parameters of [OE model](#)

```
sys = oe(data,[nb nf nk])
```

- The **bj** function estimates parameters of [BJ model](#)

```
sys = bj(data,[nb nc nd nf nk])
```

- We can create a [data object](#) by **iddata** command from the collected I/O data.

```
data = iddata(y,u,Ts)
```

Data object

Output
data

Input
data

Sample time

Variable	Description
sys	Parametric model that fits the estimated data
data	Input-output data in iddata format
na	Order of the polynomial $A(q)$
nb	Order of the polynomial $B(q)$
nc	Order of the polynomial $C(q)$
nd	Order of the polynomial $D(q)$
nf	Order of the polynomial $F(q)$
nk	Number of I/O delays of discrete-time model

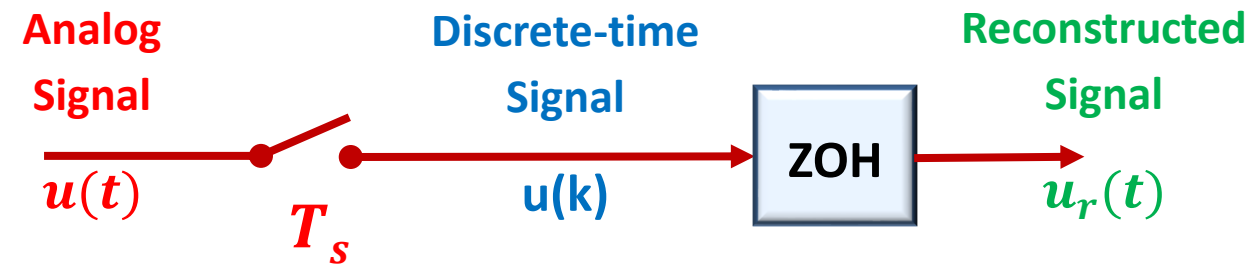
Note that the DT models must be converted to CT model.

Discrete-time TF Model Estimation in MATLAB

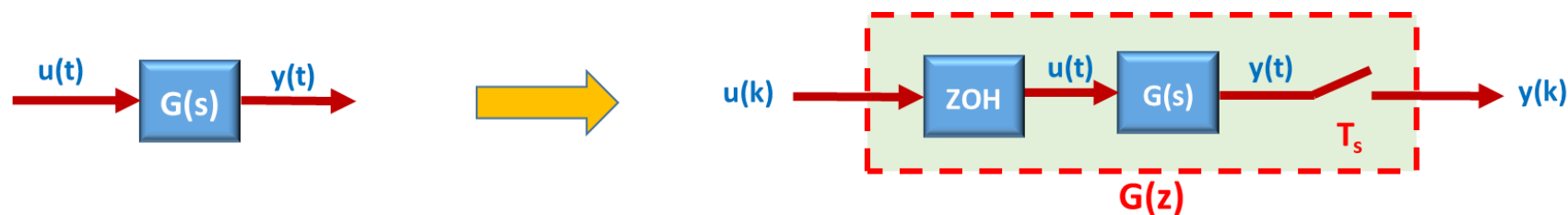
□ Sampling & Reconstruction

- **Sampling** is the process of converting the CT signal to a DT signal
- To avoid the **aliasing effect** the sampling rate must be selected based on the **Sampling Theorem**:
- There are several common techniques to reconstruct the sampled discrete-time signal.
- The simple way is using **Zero-Order-Hold (ZOH)**, which assumes the input is **piecewise constant** over the **sample time T_s** .

$$f_s \geq 2f_{max}$$

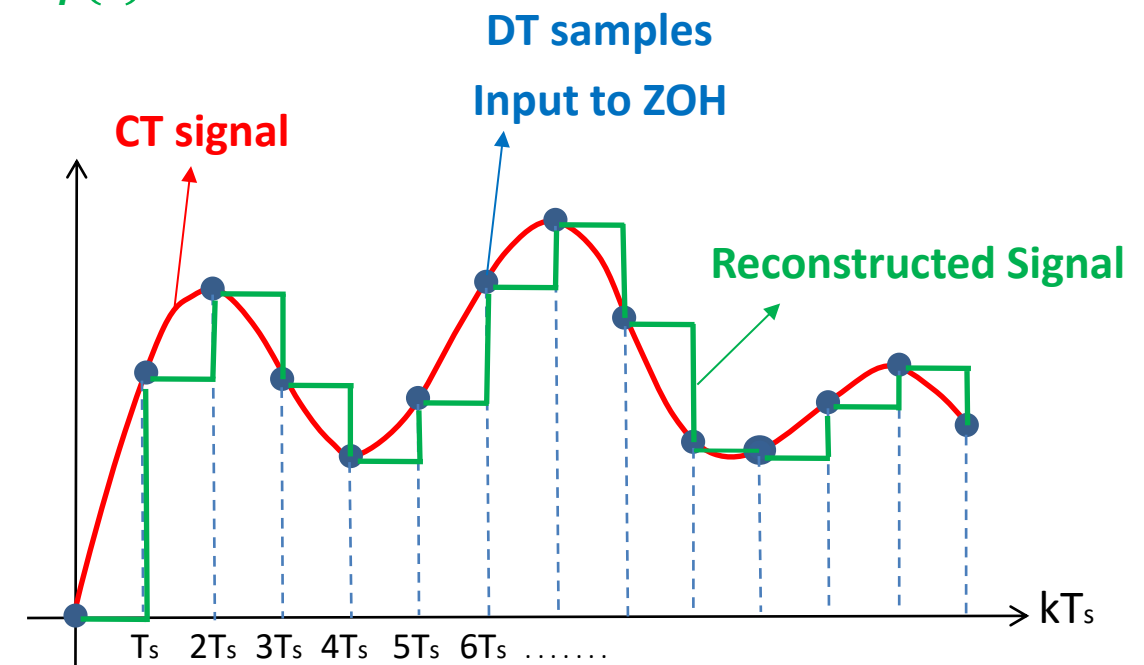


- ZOH adds **one sample delay** to the DT transfer function.



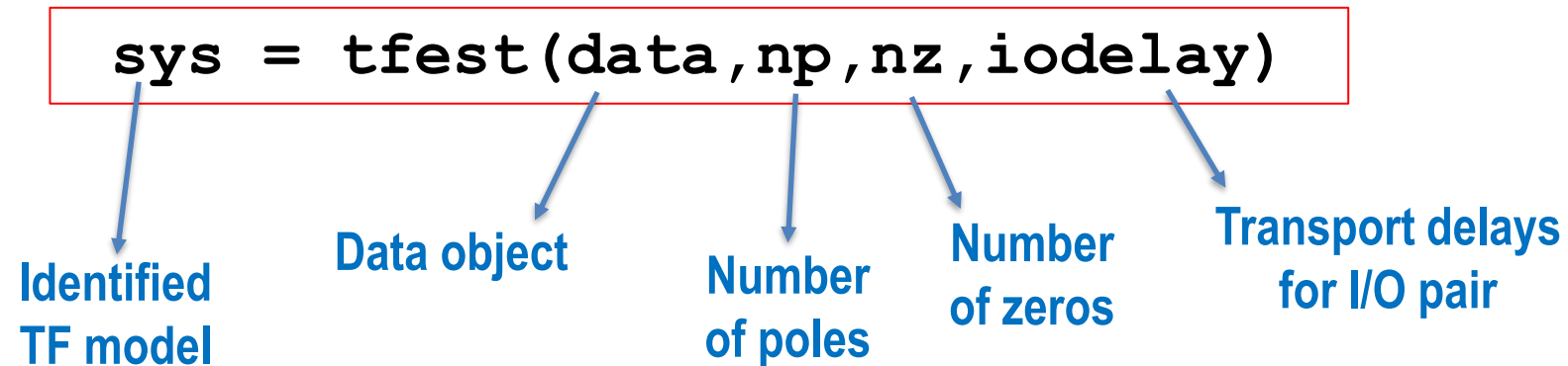
- MATLAB function to convert DT model to CT model.

$$\text{sysCT} = \text{d2c}(\text{sysDT}, \text{'zoh'})$$

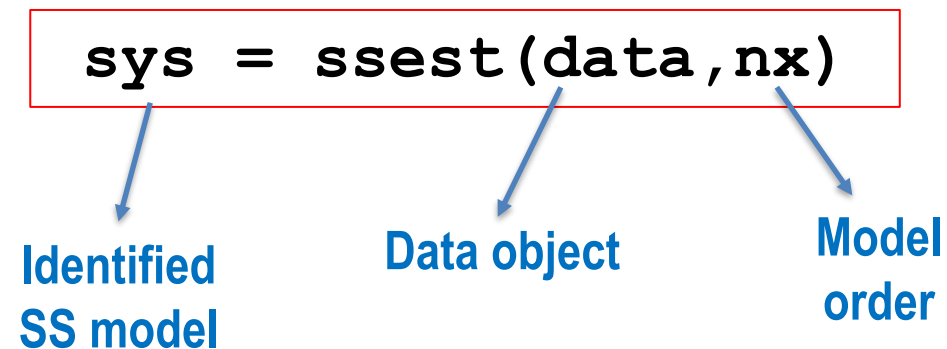


Continuous-Time Model Estimation in MATLAB

- We can directly identify the CT Transfer Function Model and State-Space Model of a system using the time-domain input and output signals. **No need to do the DT to CT conversion.**



- We can specify an unknown transport delay for the transfer function model by setting the **iodelay** to **nan**. Then MATLAB will estimate the appropriate delay-time based on the input-output data.



- We can estimate the optimal model order for state-space model by setting **nx** as a range. For example: **nx = 1:10;**

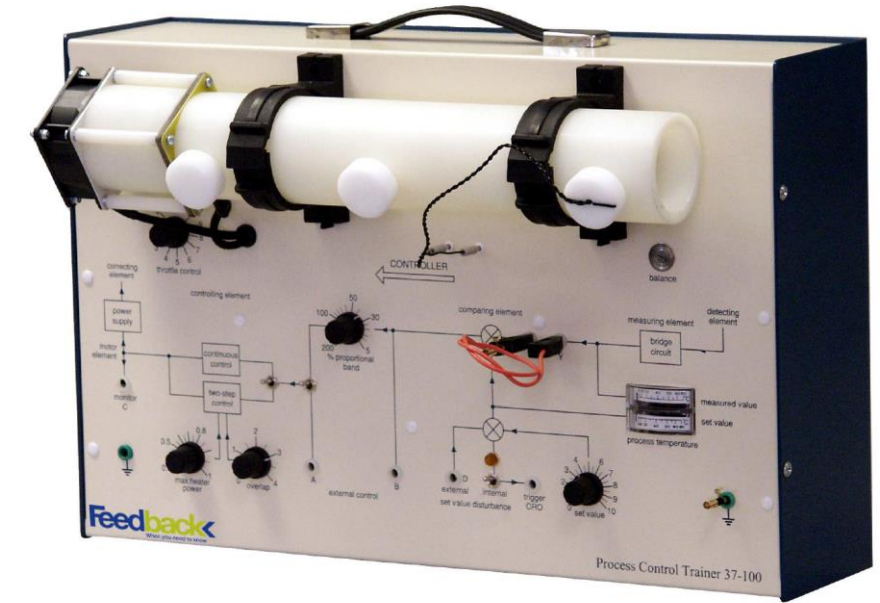
Determining Model Order and Delay

- Estimation of a model using I/O data requires selection of a **model structure** (such as DT or CT, state-space or transfer function), **model order** (e.g., number of poles and zeros) and the **I/O delays** in advance.
- The **order** and **delays** are not known in **Black-box** identification.
- We can determine the model order and delay in one of the following ways:
 - Guess their values by **visually inspecting the data** or based on the **prior knowledge of the system**.
 - **Estimate delay** as a part of transfer function model estimation in **tfest** function.
 - To estimate **delay**, you can also use one of the following tools:
 - Estimate delay using **delayest**.
 - Compute impulse response using **impulseest**.
 - To estimate **model order**, you can also use one of the following tools:
 - Select the model order in **n4sid** by specifying the model order range as a vector.
 - Choose the model order of an ARX model using **arxstruc** and **selstruc**. These command select the number of poles, zeros and delay.

Case Study: A Laboratory Scale Hairdryer

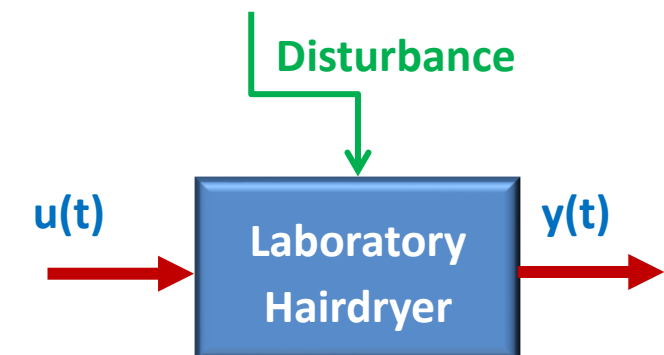
❑ Feedback's Process Control Trainer PT326

- This example shows how to develop a simple model from a real laboratory scale process data.
- The process function operates like a **hairdryer**:
 - Air is fanned through a tube and heated at the inlet
 - The **input $u(t)$** is the power of the heating device, which is a mesh of resistor wires.
 - The **output $y(t)$** is the outlet air temperature.
 - The system is **well behaved** with **simple dynamics** and **small disturbances**.
- This example shows the basic steps of a system identification procedure:
 - 1) How to import the data and apply preprocessing steps
 - 2) How to estimate model order and delay
 - 3) How to estimate DT transfer function model
 - 4) How to validate the model to the actual output data from the experiment.
 - 5) Convert the DT model to CT transfer function model.



Process Control Trainer

http://www.feedback-instruments.com/pdf/brochures/37-100_datasheet_ProcessControlTrainer_04_2013.pdf



Case Study: A Laboratory Scale Hairdryer

Step 1: Collect the I/O Data

Consider the following collected I/O data from a laboratory model of a hairdryer.

([This data is available in MATLAB System Identification Toolbox](#)) ($N = 1000$, $T_s = 0.08\text{sec}$)

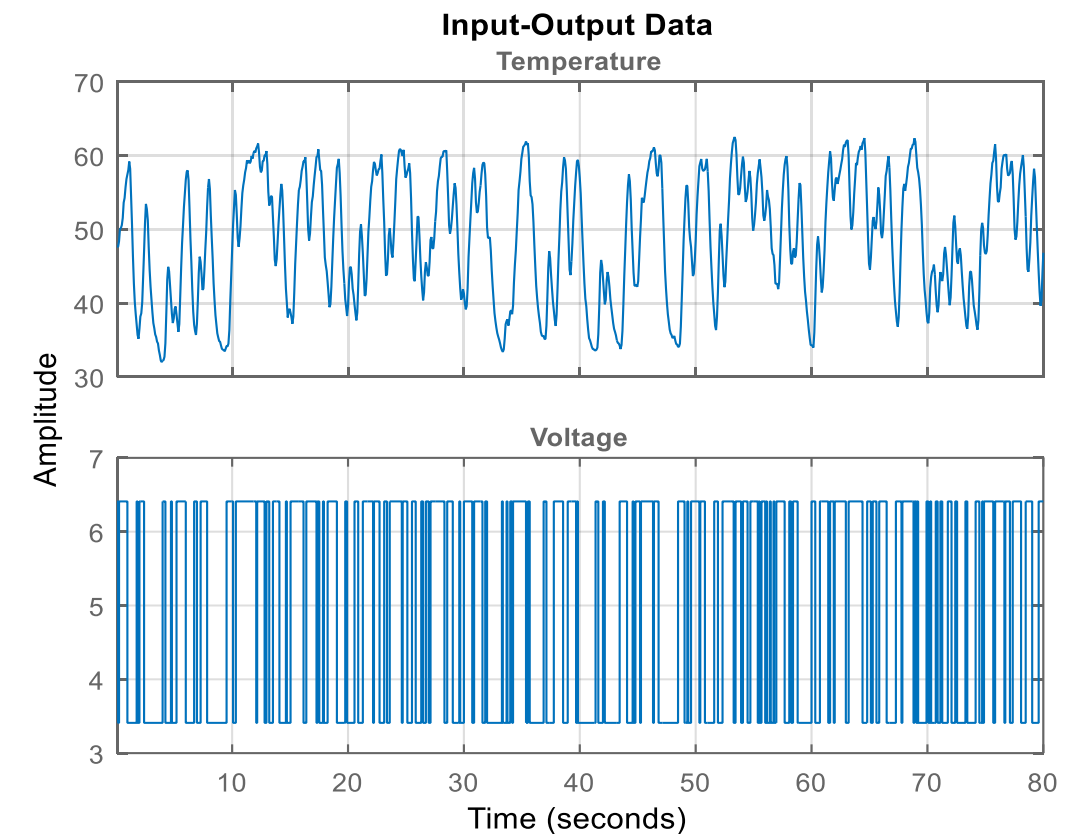
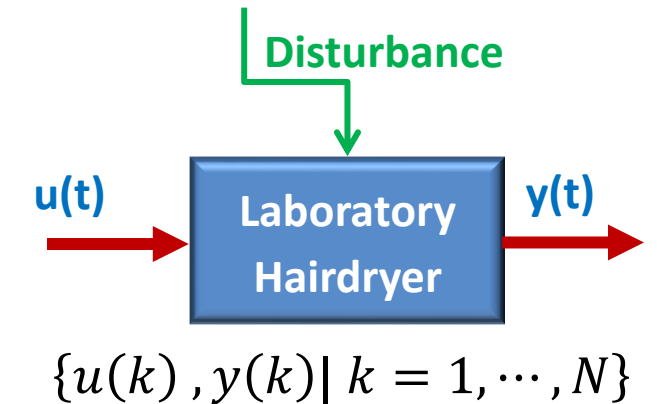
```
load dry2          ← Load I/O data from MATLAB
data = dry2(1:N);   ← I/O data object
plot(data)         ← Plot raw I/O data
```

Input and output signals are defined as follows:

Input: The voltage over the heating device, which is a mesh of resistor wires.

Output: The outlet air temperature represented by the measured thermocouple voltage.

- Here, the process has a simple dynamics with quite small disturbances
- The collected data has good SNR.

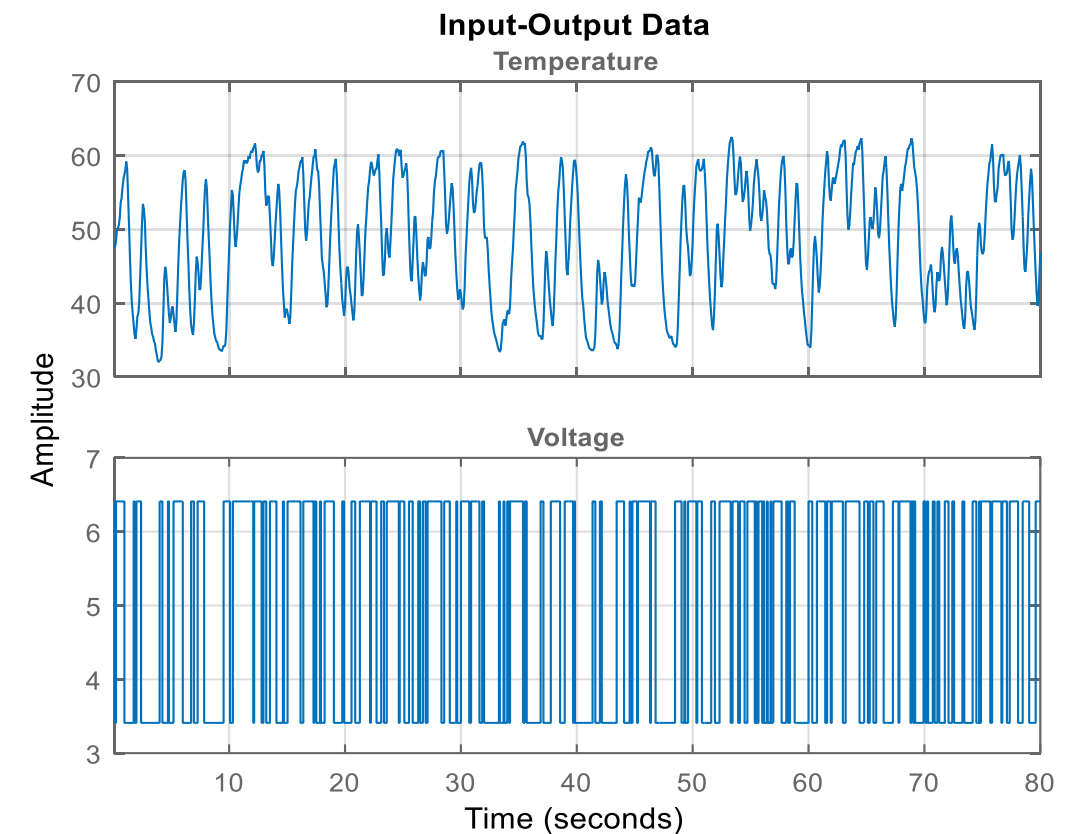
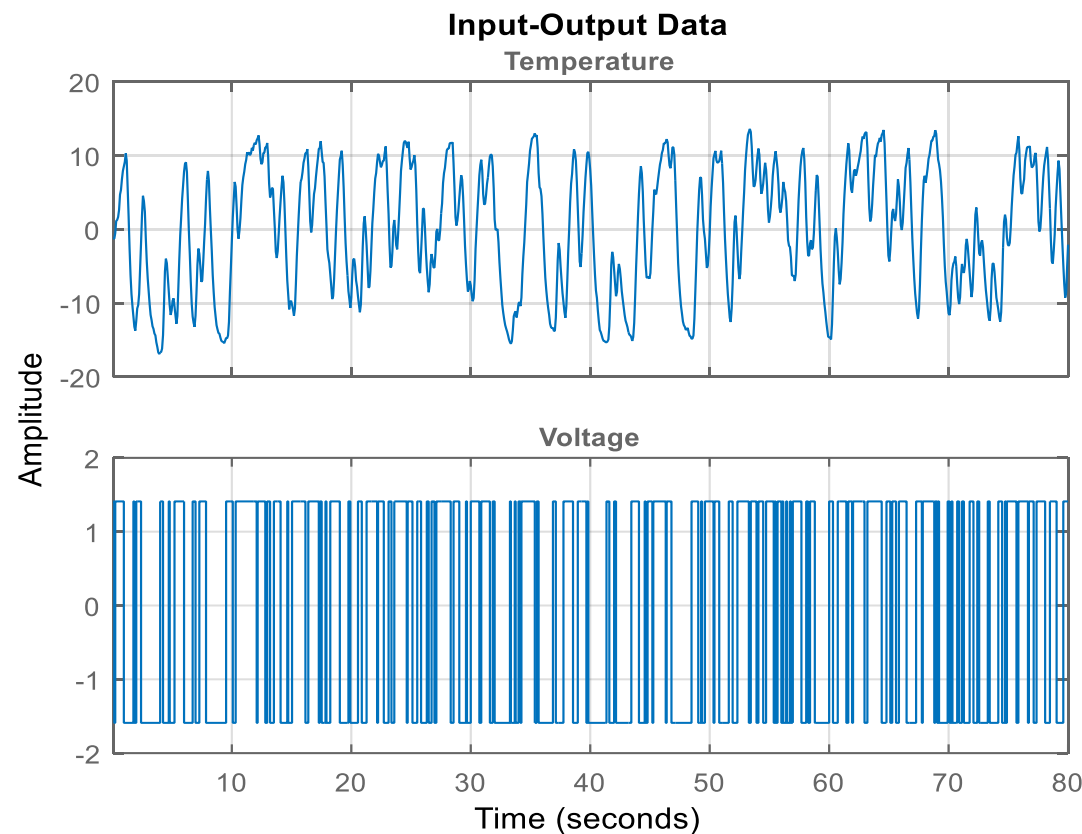
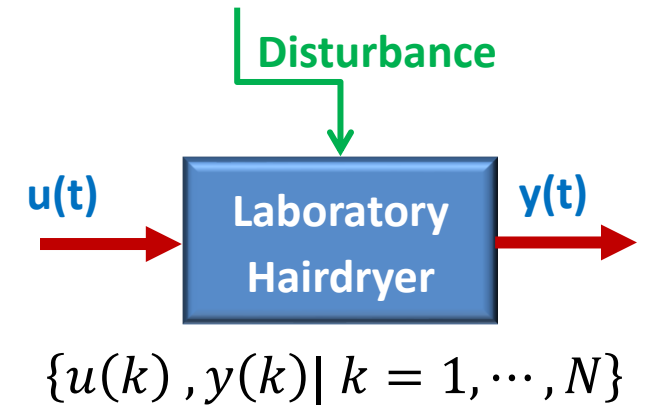


Case Study: A Laboratory Scale Hairdryer

Step 2: Examine the I/O Data (Outliers, Trends and Aliasing effects)

From the data plot, it can be seen that the data has **non-zero mean (DC-offset)**.
Therefore, first we have to remove the non-zero mean from the I/O data

```
data = dtrend(data); ← Remove non-zero mean value
```



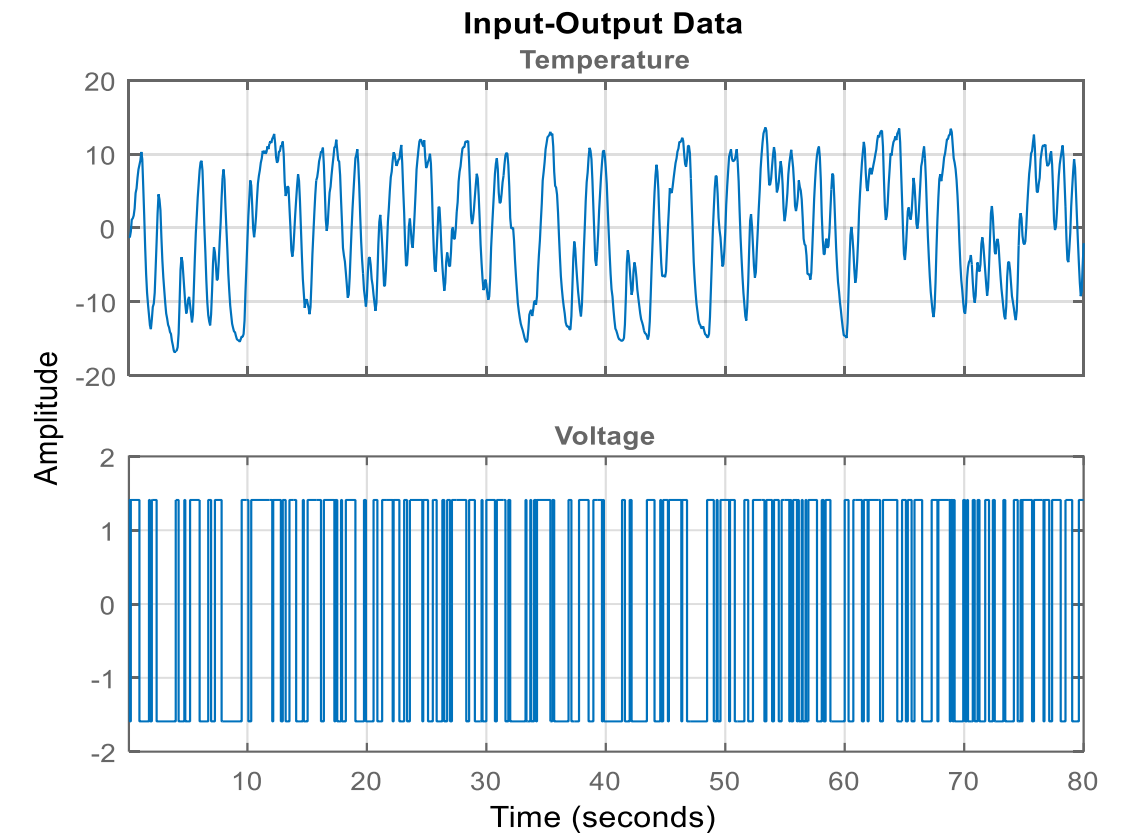
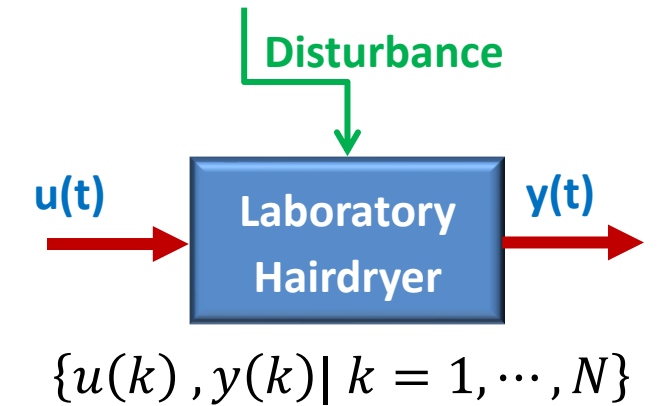
Case Study: A Laboratory Scale Hairdryer

Step 3: Split the I/O data to the Identification data and Validation data

Here, we have $N = 1000$ samples of I/O data. The first 700 samples are used for identification (estimation the model parameters), the rest for validation.

$$N_{id} = 700, \quad N_{val} = 300$$

```
datai = data(1:700);      ← Identification data  
datav = data(701:N);     ← Validation data
```



Case Study: A Laboratory Scale Hairdryer

Step 4: Obtain some priori information about the system order and time-delay

□ Delay Estimation:

Here, we use the `delayest` and `impulseest` functions to estimate the delay.

```
delay = delayest(data)      ← Delay estimation
```

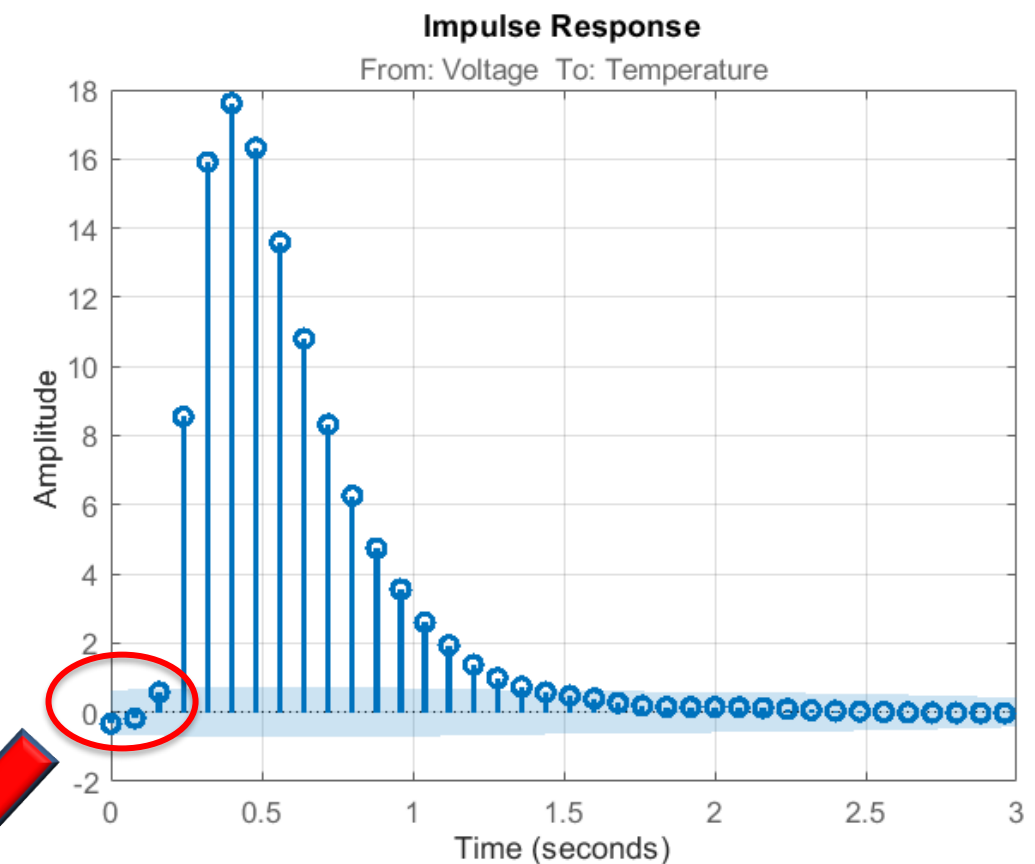
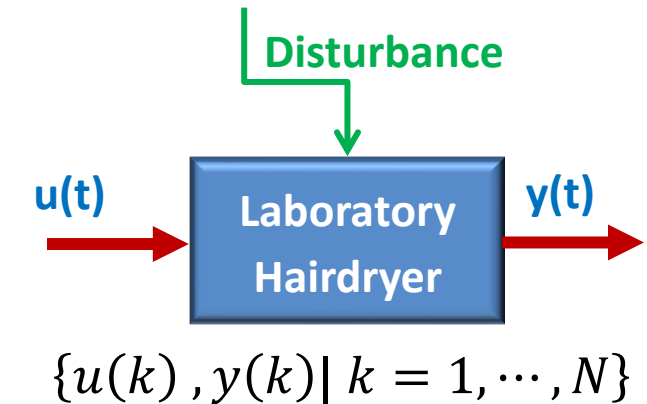
```
delay =
```

```
3
```

```
ir = impulseest(data);      ← Impulse response estimation  
ir_plot = impulseplot(ir);  ← Impulse response plot  
showConfidence(ir_plot,5)   ← Show confidence intervals
```

- We plot the impulse response with a confidence interval represented by 5 standard deviations.
- The results show that we can consider three sample delays, $n_k = 3$.
- Note that, the number of delay samples includes the one delay sample for ZOH.

$$n_k = 3$$



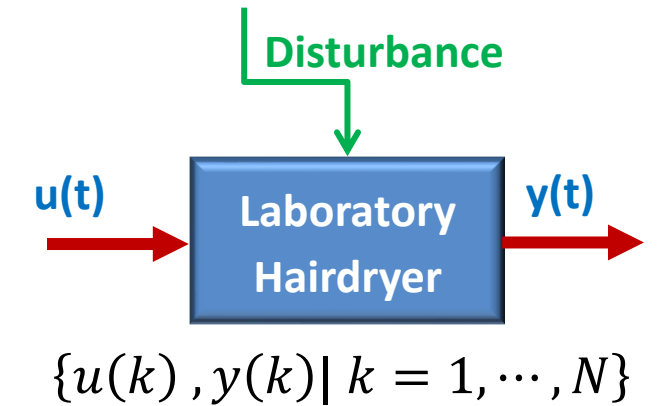
Case Study: A Laboratory Scale Hairdryer

Step 4: Obtain some priori information about the system order and time-delay

□ Order Estimation:

Here, we use the `n4sid` function to estimate the order from state-space structure.

```
n4sid(data,1:10,'InputDelay',2); ← Order estimation
```



- We considered the range of orders from 1 to 10.
- The "`InputDelay`" was set to 2 because by default `n4sid` estimates a model accounting for one sample lag between input and output.
- The default **order**, indicated in the figure, is **two**.

Case Study: A Laboratory Scale Hairdryer

Step 5: Select the parametric model structure and estimate the model

OE Model Estimation:

```
nb = 2;           ← Order of the B(q) polynomial
nf = 2;           ← Order of the F(q) polynomial
nk = 3;           ← Number of samples delay
M_OE = oe(datai,[nb nf nk]); ← OE model from identification data
```

```
M_OE =
Discrete-time OE model: y(t) = [B(z)/F(z)]u(t) + e(t)
  B(z) = 0.6795 z^-3 + 0.4541 z^-4
  F(z) = 1 - 1.274 z^-1 + 0.3946 z^-2
```

Sample time: 0.08 seconds

Parameterization:

Polynomial orders: nb=2 nf=2 nk=3

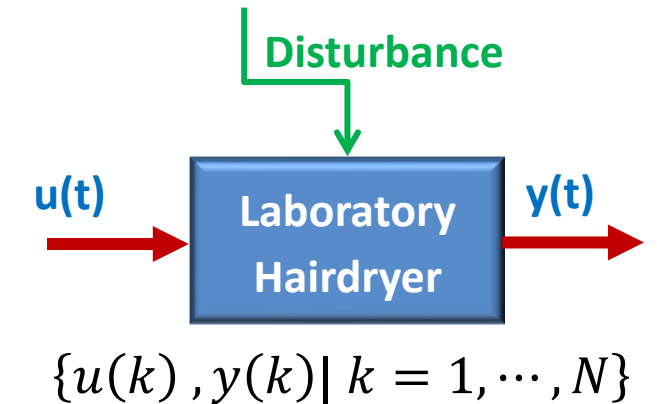
Number of free coefficients: 4

Status:

Estimated using OE on time domain data "dry2".

Fit to estimation data: 88.76%

FPE: 0.9316, MSE: 0.9211



- The OE model estimation

$$y(k) = \frac{B(q)}{F(q)} u(k) + e(k)$$

$$y(k) = \frac{0.6795q^{-3} + 0.4541q^{-4}}{1 - 1.274q^{-1} + 0.3946q^{-2}} u(k) + e(k)$$

$$y(k) = \frac{0.6795 + 0.4541q^{-1}}{1 - 1.27q^{-1} + 0.3946q^{-2}} u(k - 3) + e(k)$$

Case Study: A Laboratory Scale Hairdryer

Step 5: Select the parametric model structure and estimate the model

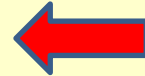
□ ARX Model Estimation:

```
na = 2;           ← Order of the A(q) polynomial
nb = 2;           ← Order of the B(q) polynomial
nk = 3;           ← Number of samples delay
M_ARX = arx(datai,[na nb nk]); ← ARX model from identification data
```

M_ARX =
Discrete-time ARX model: $A(z)y(t) = B(z)u(t) + e(t)$

$$A(z) = 1 - 1.288z^{-1} + 0.4053z^{-2}$$

$$B(z) = 0.6552z^{-3} + 0.4324z^{-4}$$



Sample time: 0.08 seconds

Parameterization:

Polynomial orders: na=2 nb=2 nk=3

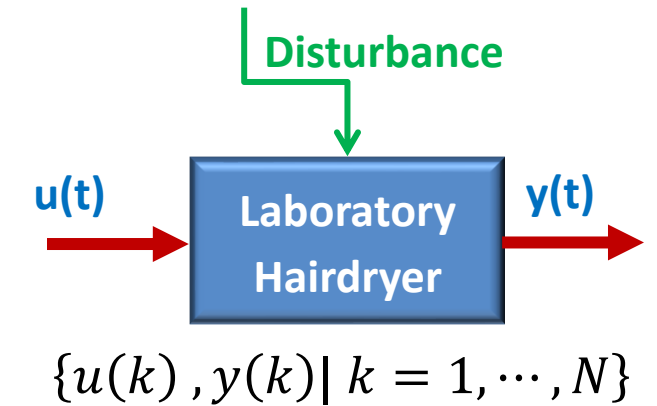
Number of free coefficients: 4

Status:

Estimated using ARX on time domain data "dry2".

Fit to estimation data: 95.32%

FPE: 0.1627, MSE: 0.16



- The ARX model estimation

$$y(k) = \frac{B(q)}{A(q)}u(k) + \frac{1}{A(q)}e(k)$$

- The ARX model has better fit to estimation data.

$$y(k) = \frac{0.6552 + 0.4324q^{-1}}{1 - 1.288q^{-1} + 0.4053q^{-2}}u(k-3) + \frac{1}{1 - 1.288q^{-1} + 0.4053q^{-2}}e(k)$$

Case Study: A Laboratory Scale Hairdryer

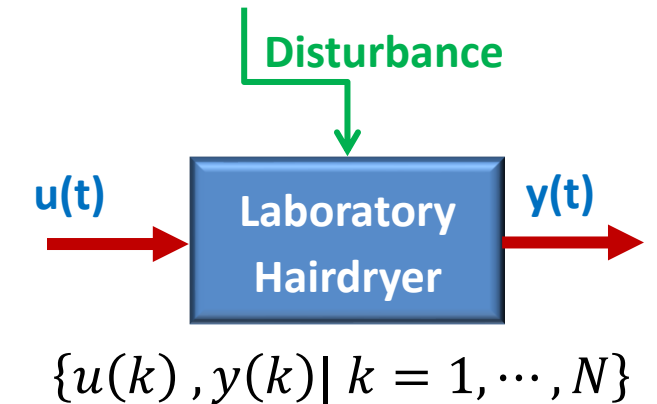
Step 6: Model Validation

□ OE Model Validation:

Here, we use the `present` function in MATLAB to validate the OE model parameters estimation.

```
present(M_OE)           ← Display model information
```

```
M_OE =  
Discrete-time OE model:  $y(t) = [B(z)/F(z)]u(t) + e(t)$   
    B(z) = 0.6795 (+/- 0.03021) z^-3 + 0.4541 (+/- 0.05239) z^-4  
    F(z) = 1 - 1.274 (+/- 0.0253) z^-1 + 0.3946 (+/- 0.0228) z^-2  
  
Sample time: 0.08 seconds  
Parameterization:  
    Polynomial orders:    nb=2    nf=2    nk=3  
    Number of free coefficients: 4  
  
Status:  
Estimated using OE on time domain data "dry2".  
Fit to estimation data: 88.76%  
FPE: 0.9316, MSE: 0.9211
```



- Confidence intervals of the estimation
- Fit to estimation data
- Mean Square Error (MSE)
- Final Prediction Error (FPE)

Case Study: A Laboratory Scale Hairdryer

Step 6: Model Validation

□ ARX Model Validation:

Here, we use the `present` function in MATLAB to validate the ARX model parameters estimation.

```
present(M_ARX)           ← Display model information
```

```
M_ARX =  
Discrete-time ARX model: A(z)y(t) = B(z)u(t) + e(t)  
  A(z) = 1 - 1.288 (+/- 0.01325) z^-1 + 0.4053 (+/- 0.01211) z^-2  
  B(z) = 0.6552 (+/- 0.01326) z^-3 + 0.4324 (+/- 0.02026) z^-4
```

```
Sample time: 0.08 seconds
```

```
Parameterization:
```

```
Polynomial orders:   na=2   nb=2   nk=3
```

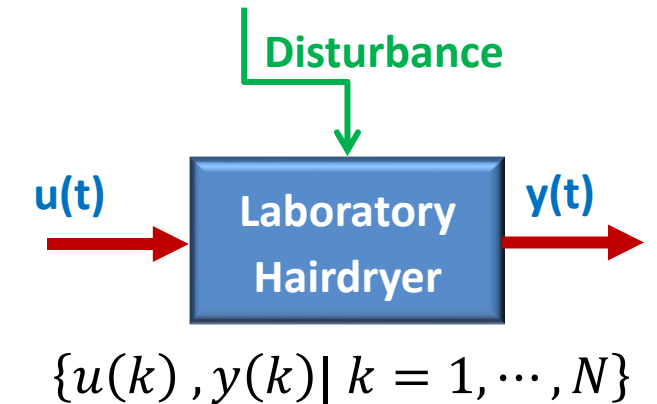
```
Number of free coefficients: 4
```

```
Status:
```

```
Estimated using ARX on time domain data "dry2".
```

```
Fit to estimation data: 95.32%
```

```
FPE: 0.1627, MSE: 0.16
```



- Confidence intervals of the estimation
- Fit to estimation data
- Mean Square Error (MSE)
- Final Prediction Error (FPE)

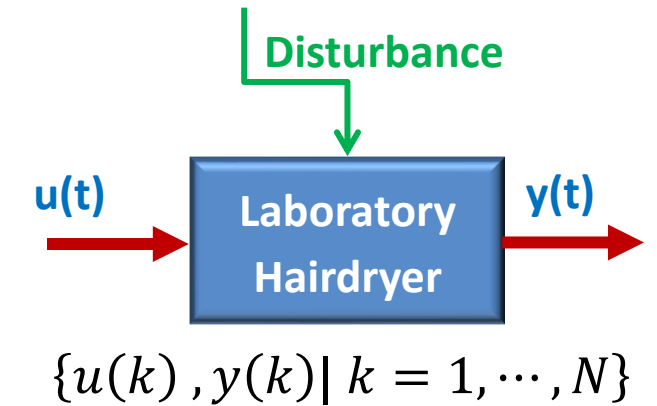
- We can see that for ARX model estimation the **FPE** and **MSE** criteria give a better results than the OE model.

Case Study: A Laboratory Scale Hairdryer

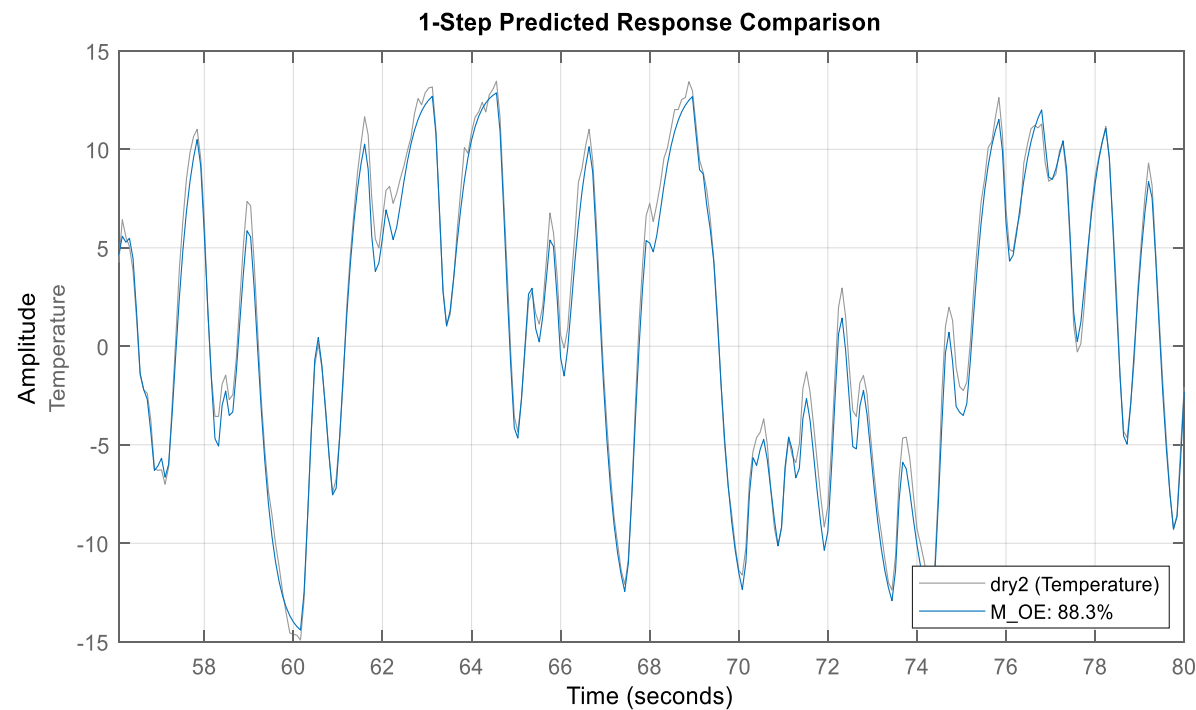
Step 6: Model Validation

Here, we use the `compare` function in MATLAB to validate the model parameters estimation.

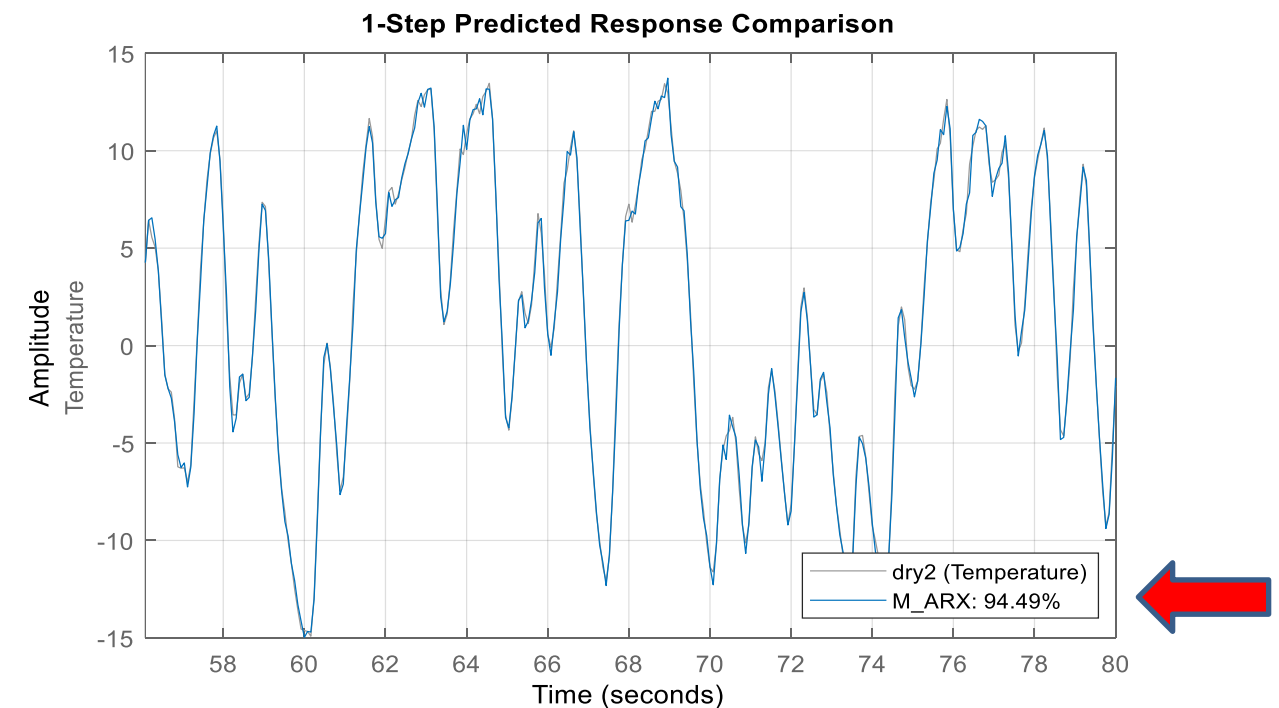
```
compare(datav,M_OE,1)      ← Compare OE model by validation data  
compare(datav,M_ARX,1)    ← Compare ARX model by validation data
```



□ OE Model Validation:



□ ARX Model Validation:



- Since the ARX model shows better fit to the validation data, we can conclude that the data is more corrupted by **process noise** than the **white noise**.

Case Study: A Laboratory Scale Hairdryer

Step 7: Determine the final DT and CT models

The ARX Model:

$$y(k) = \frac{0.6552 + 0.4324q^{-1}}{1 - 1.288q^{-1} + 0.4053q^{-2}} u(k-3) + \frac{1}{1 - 1.288q^{-1} + 0.4053q^{-2}} e(k)$$

- Convert DT model to CT model using **d2c** command in MATLAB:

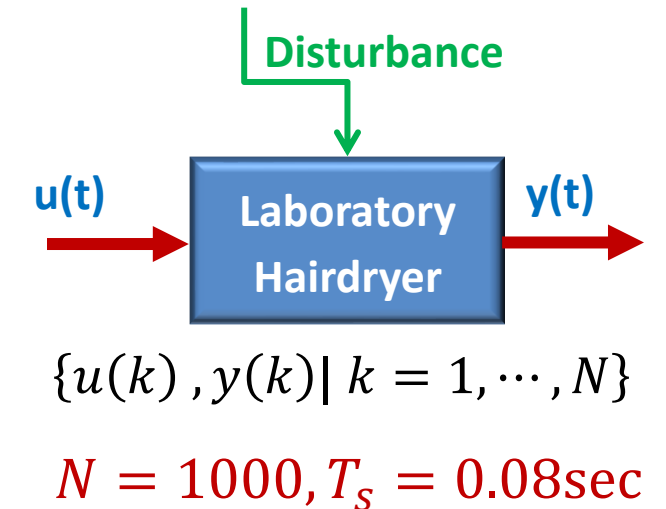
```
mc_arx = d2c(M_ARX, 'zoh') ← convert DT ARX model to CT model G(s)
```

```
mc_arx =  
Continuous-time ARMAX model: A(s)y(t) = B(s)u(t) + C(s)e(t)  
A(s) = s^2 + 11.29 s + 28.2  
B(s) = 0.5902 s + 261.9  
C(s) = s^2 + 26.56 s + 240.8  
  
Input delays (listed by channel): 0.16
```

$$T_d = (n_k - 1)T_s = 2 \times 0.08 = 0.16\text{sec}$$

The CT Transfer Function Model

$$G(s) = e^{-0.16s} \frac{0.5902s + 261.9}{s^2 + 11.29s + 28.2}$$

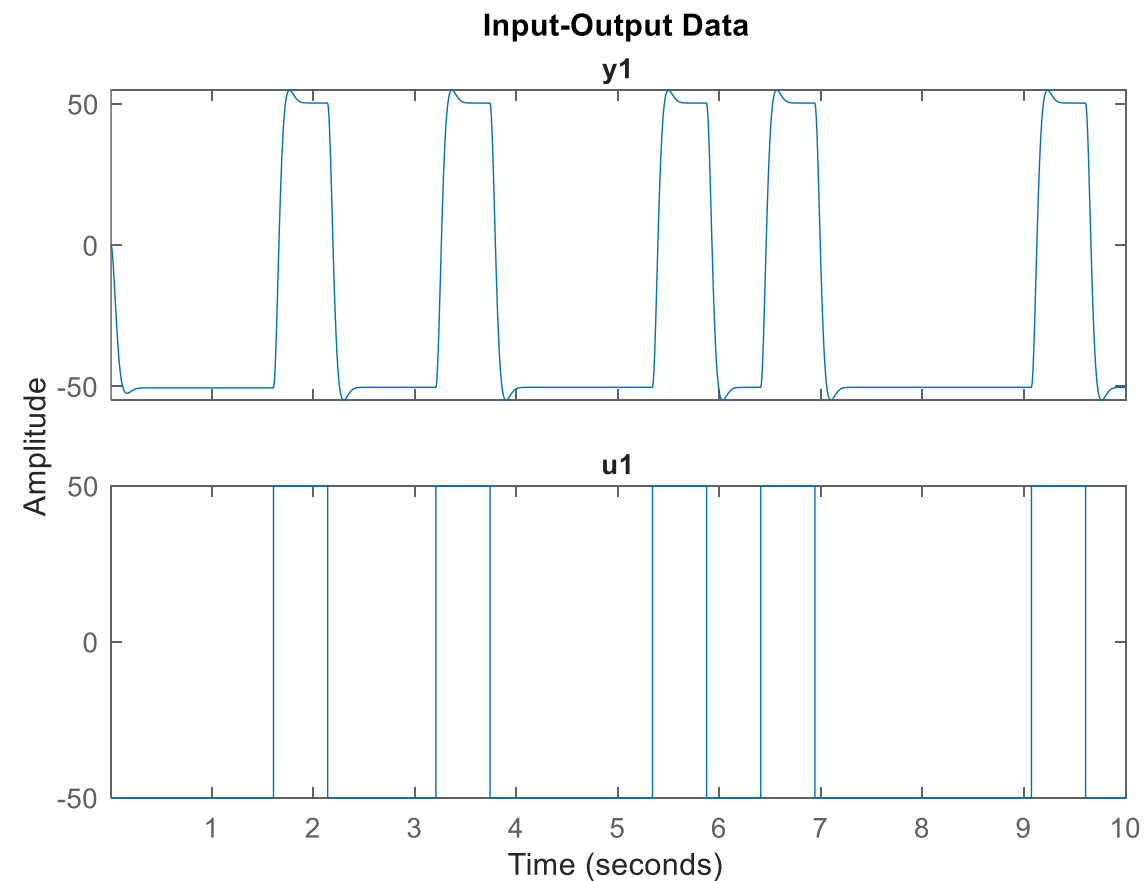


Identification of CT Transfer Function Models

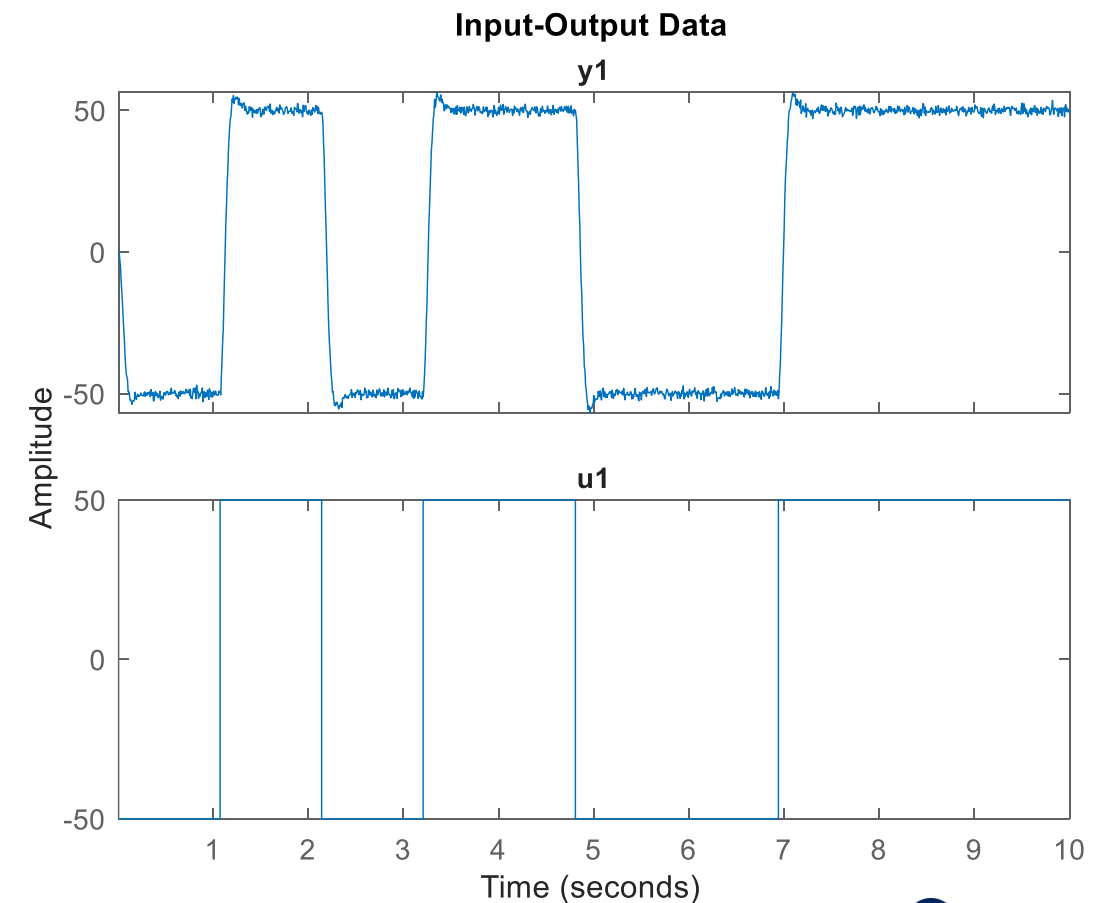
Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

Assume that two sets of **noisy** and **noise-free** data has been collected from a closed-loop DC motor position control system. **Input** is the reference angular position in degrees; **Output** is the angular displacement of the DC motor.

Noise-free Dataset



Noisy Dataset



Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

Step 1: Collect and Examine the I/O Data for Outliers and Trends

```
Ts = 0.0067;           % Sampling time

data = iddata(y,u,Ts);  % Noise-free I/O data object
plot(data)             % Plot noise-free I/O data

datan = iddata(yn,un,Ts); % Noise-free I/O data object
plot(datan)            % Plot raw I/O data
```

Step 2: Split the I/O Data to the Identification Data and Validation Data

- Here, we have $N = 1500$ samples of I/O data. The first 1000 samples are used for **identification** (estimation the model parameters), the rest for **validation**.

```
datai = data(1:1000);   % Noise-free identification data
datav = data(1001:end); % Noise-free validation data

datani = datan(1:1000); % Noisy identification data
datanv = datan(1001:end); % Noisy validation data
```


Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

Noise-free Data

Step 3: Select the parametric model structure, estimate the model.

Here, we can consider both **second-order** and **third-order** transfer function models to compare.

□ Second-order TF Model Estimation:

```
np = 2;  
nz = 1  
sys_TF2 = tfest(datai,np,nz,nan);
```

```
sys_TF2 =  
From input "u1" to output "y1":  
  -1.598 s + 798.4  
-----  
s^2 + 39.4 s + 792.2
```



Continuous-time identified transfer function.
Parameterization:
Number of poles: 2 Number of zeros: 1
Number of free coefficients: 4

$$G(s) = \frac{-1598s + 798.4}{s^2 + 39.4s + 792.2}$$

□ Third-order TF Model Estimation:

```
np = 3;  
nz = 1  
sys_TF3 = tfest(datai,np,nz,nan);
```

```
sys_TF3 =  
From input "u1" to output "y1":  
  538.1 s + 9.014e04  
-----  
s^3 + 148 s^2 + 5170 s + 8.94e04
```



Continuous-time identified transfer function.
Parameterization:
Number of poles: 3 Number of zeros: 1
Number of free coefficients: 5

$$G(s) = \frac{538.1s + 90140}{s^3 + 148s^2 + 5170s + 89400}$$

Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

Noise-free Data

Step 4: Validate the model.

Here, we use the `present` function in MATLAB to validate the TF model parameters estimation.

□ Second-order TF Model Validation:

```
present(sys_TF2)
```

```
Sys_TF2 =
```

From input "u1" to output "y1":

-1.598 (+/- 0.06281) s + 798.4 (+/- 2.801)

s^2 + 39.4 (+/- 0.1177) s + 792.2 (+/- 2.721)

Parameterization:

Number of poles: 2 Number of zeros: 1

Number of free coefficients: 4

Estimated using TFEST on time domain data "data1"

Fit to estimation data: 99.62%

FPE: 0.028, MSE: 0.02766

□ Third-order TF Model Validation:

```
present(sys_TF3)
```

```
sys_TF3 =
```

From input "u1" to output "y1":

538.1 (+/- 18) s + 9.014e04 (+/- 8751)

s^3 + 148 (+/- 11.1) s^2 + 5170 (+/- 431.3) s + 8.94e04 (+/- 8710)

Parameterization:

Number of poles: 3 Number of zeros: 1

Number of free coefficients: 5

Estimated using TFEST on time domain data "data1".

Fit to estimation data: 99.71%

FPE: 0.01566, MSE: 0.01541

- The **third-order** model shows better fit to estimation data, but the **parameter confidence intervals are large**.

Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

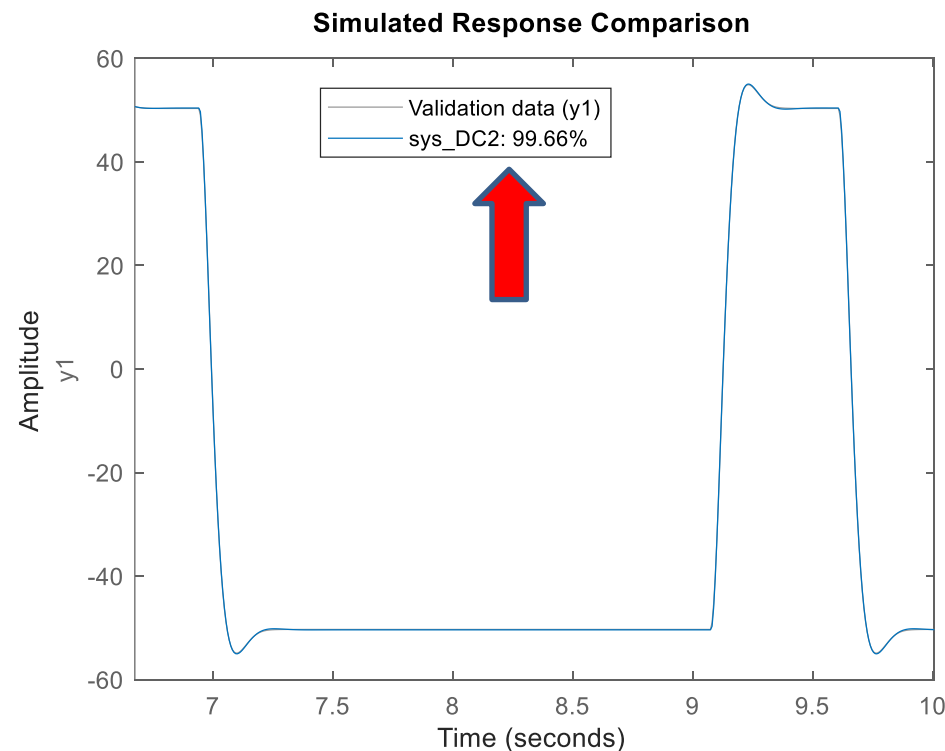
Noise-free Data

Step 4: Validate the model.

Here, we use the `compare` function in MATLAB to validate the estimated TF model using the validation dataset.

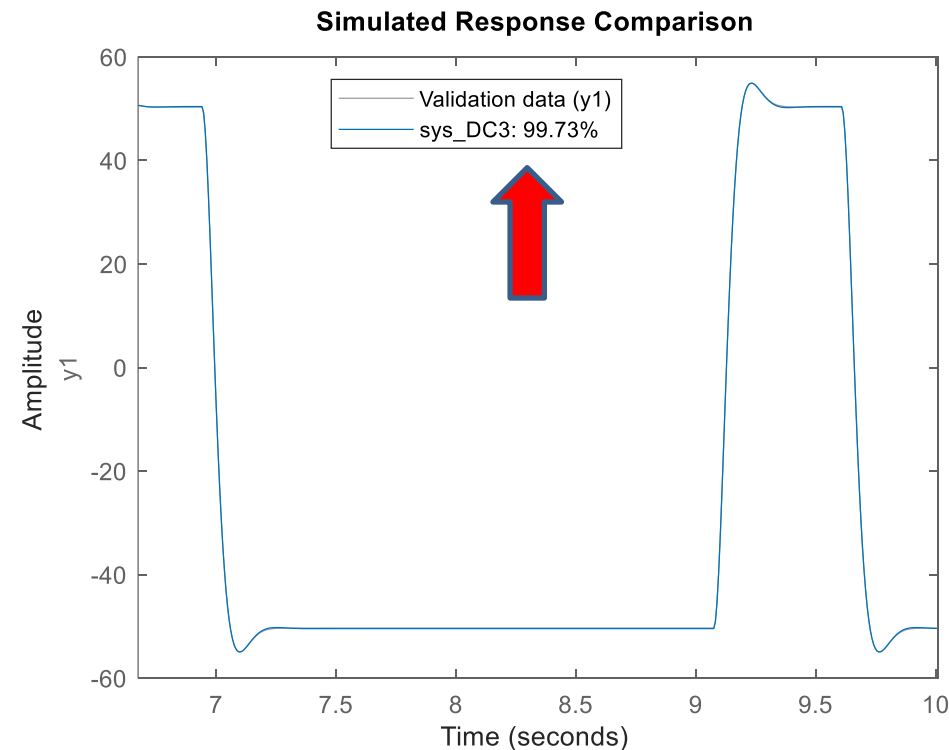
❑ Second-order TF Model Validation:

```
compare(sys_TF2,datav,1)
```



❑ Third-order TF Model Validation:

```
compare(sys_TF3,datav,1)
```



- The **third-order** model gives slightly better results. However, there is no significant difference between the models.

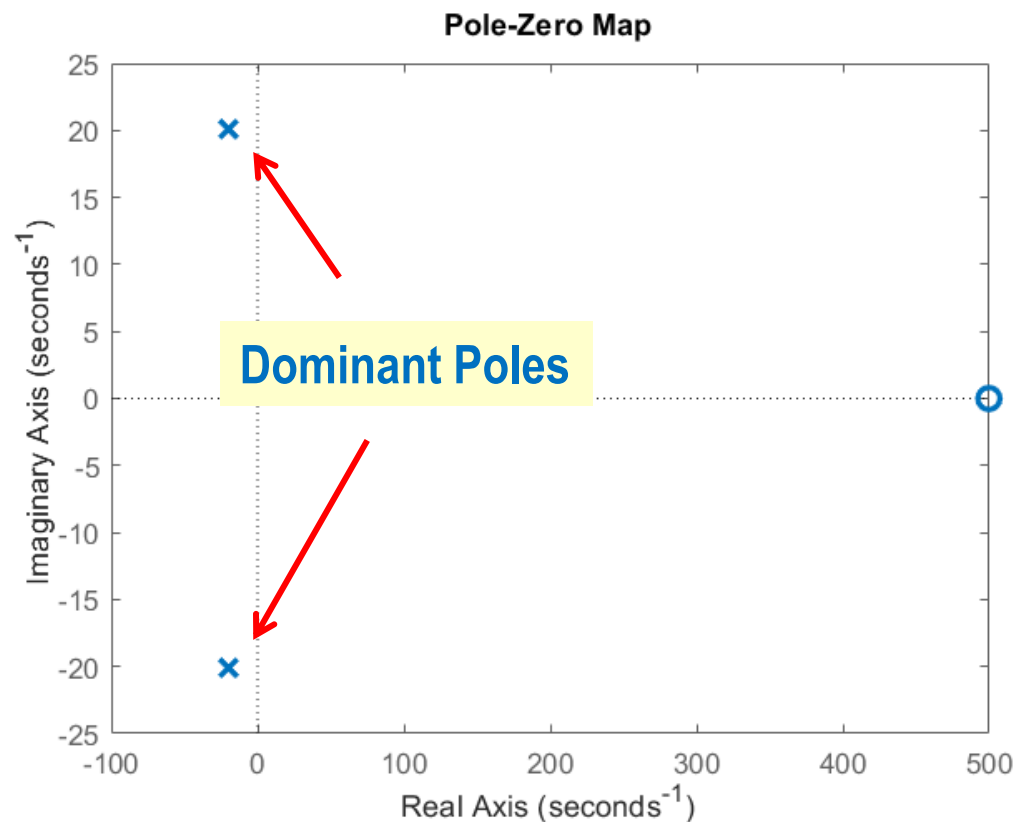
Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

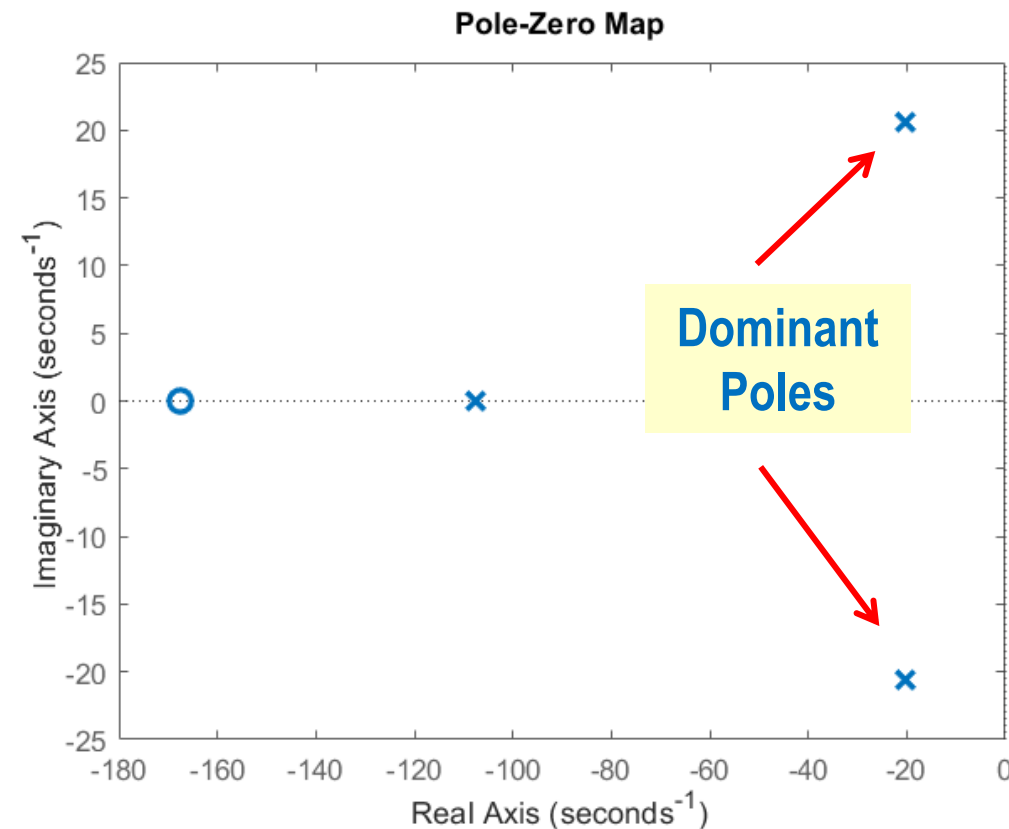
Noise-free Data

Step 5: Check the pole/zero locations for $G(s)$

☐ Second-order TF Model:



☐ Third-order TF Model:



- The pole/zero plots represent that system can be estimated as a second-order TF model with no zeroes.

Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

Noise-free Data

Step 6: Modify the Model Order and Estimate and Validate the Model

□ Second-order TF Model with No Zero Estimation:

```
np = 2;  
nz = 0  
sys_TF = tfest(datai,np,nz,nan);
```

```
sys_TF =  
  From input "u1" to output "y1":  
      740.4  
-----  
s^2 + 37.5 s + 734.6
```

Continuous-time identified transfer function.
Parameterization:
Number of poles: 2 Number of zeros: 0
Number of free coefficients: 3

$$G(s) = \frac{740.4}{s^2 + 37.5s + 734.6}$$

```
sys_TF =
```

```
  From input "u1" to output "y1":  
      740.4 (+/- 2.556)
```

```
-----  
s^2 + 37.5 (+/- 0.1368) s + 734.6 (+/- 2.437)
```

Parameterization:

Number of poles: 2 Number of zeros: 0

Number of free coefficients: 3

Fit to estimation data: 99.24%

FPE: 0.1086, MSE: 0.1075

Estimated using TFEST on time domain data "datai".

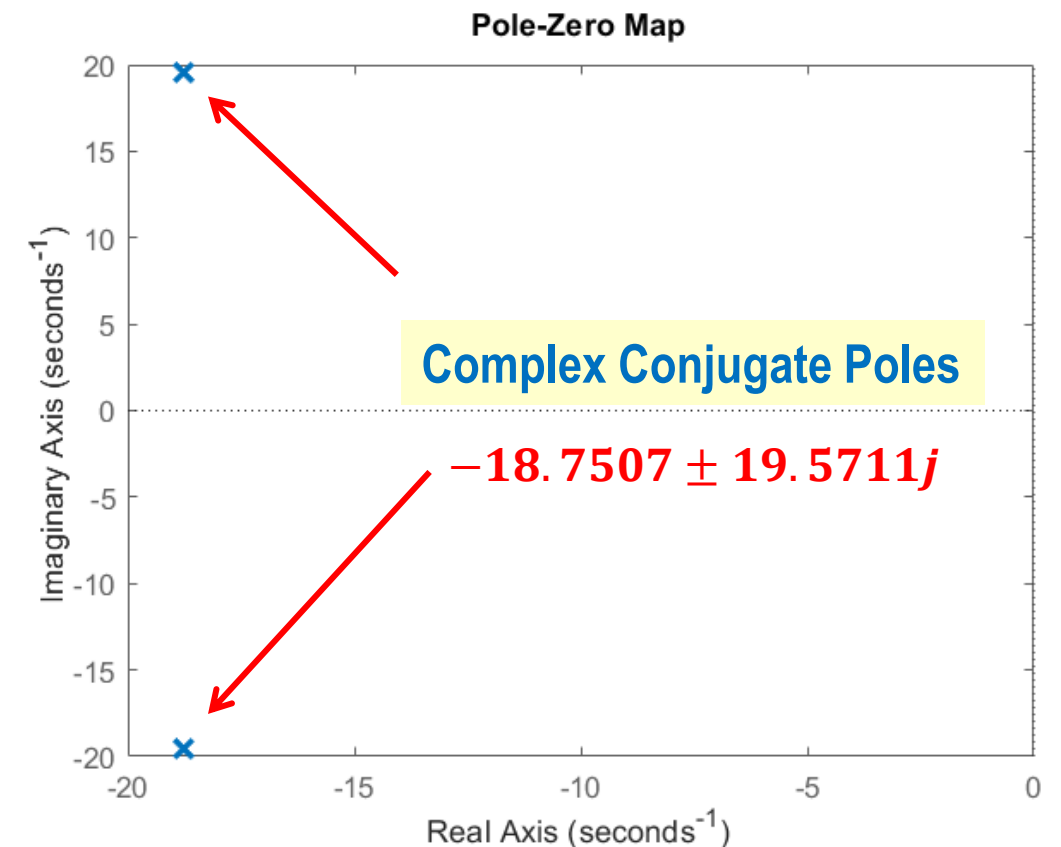
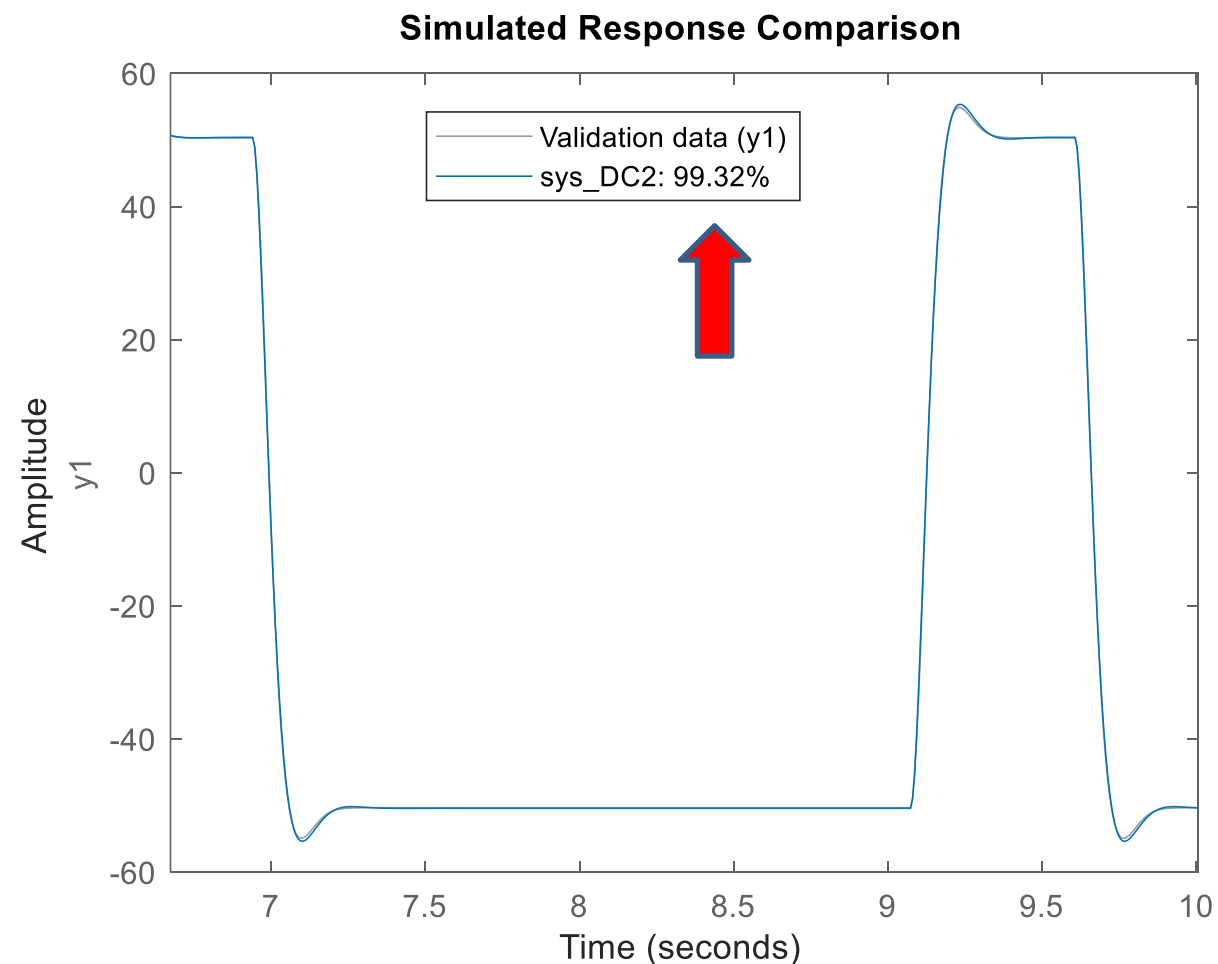
Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

Noise-free Data

Step 6: Modify the Model Order and Estimate and Validate the Model

□ Second-order TF Model with No Zero Validation:



Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

Noisy Data

Step 3: Select the parametric model structure, estimate and validate the model.

Here, we can consider a **second-order** transfer function with no zero to compare with the noise-free case.

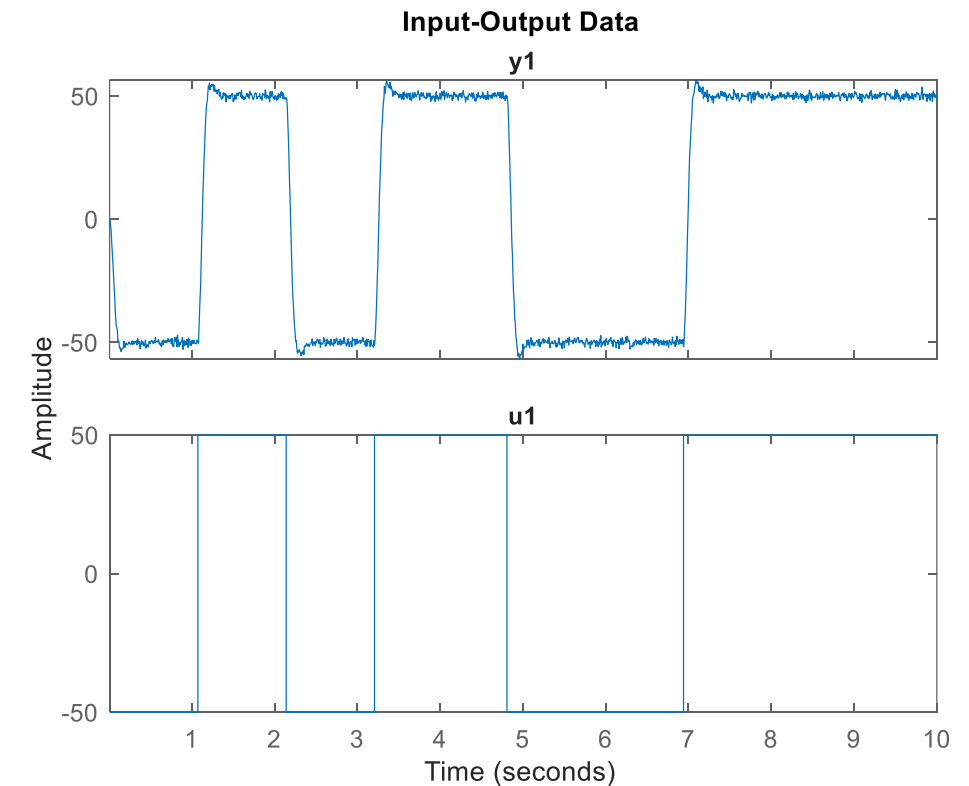
□ Second-order TF Model Estimation:

```
np = 2;  
nz = 0  
sysn_TF = tfest(datani,np,nz);
```

```
sysn_TF =  
From input "u1" to output "y1":  
      733.1  
-----  
s^2 + 36.96 s + 732.9
```

Continuous-time identified transfer function.
Parameterization:
Number of poles: 2 Number of zeros: 0
Number of free coefficients: 3

Estimated using TFEST on time domain data "datani"
Fit to estimation data: 97.85%
FPE: 1.085, MSE: 1.074



$$G(s) = \frac{733.1}{s^2 + 36.96s + 732.9}$$

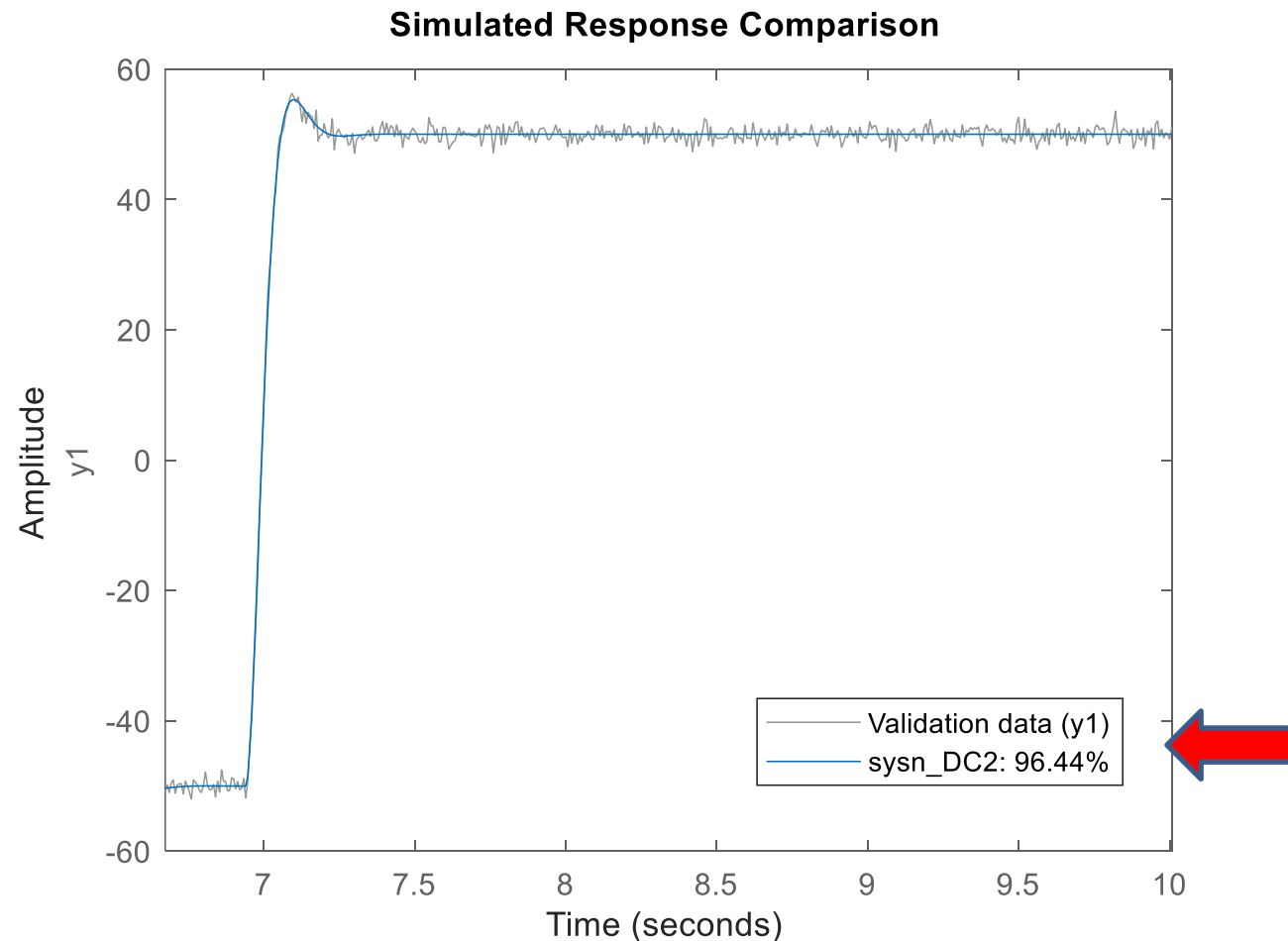
Identification of CT Transfer Function Models

Example 2 This example shows how to develop a simple CT transfer function model from input-output data.

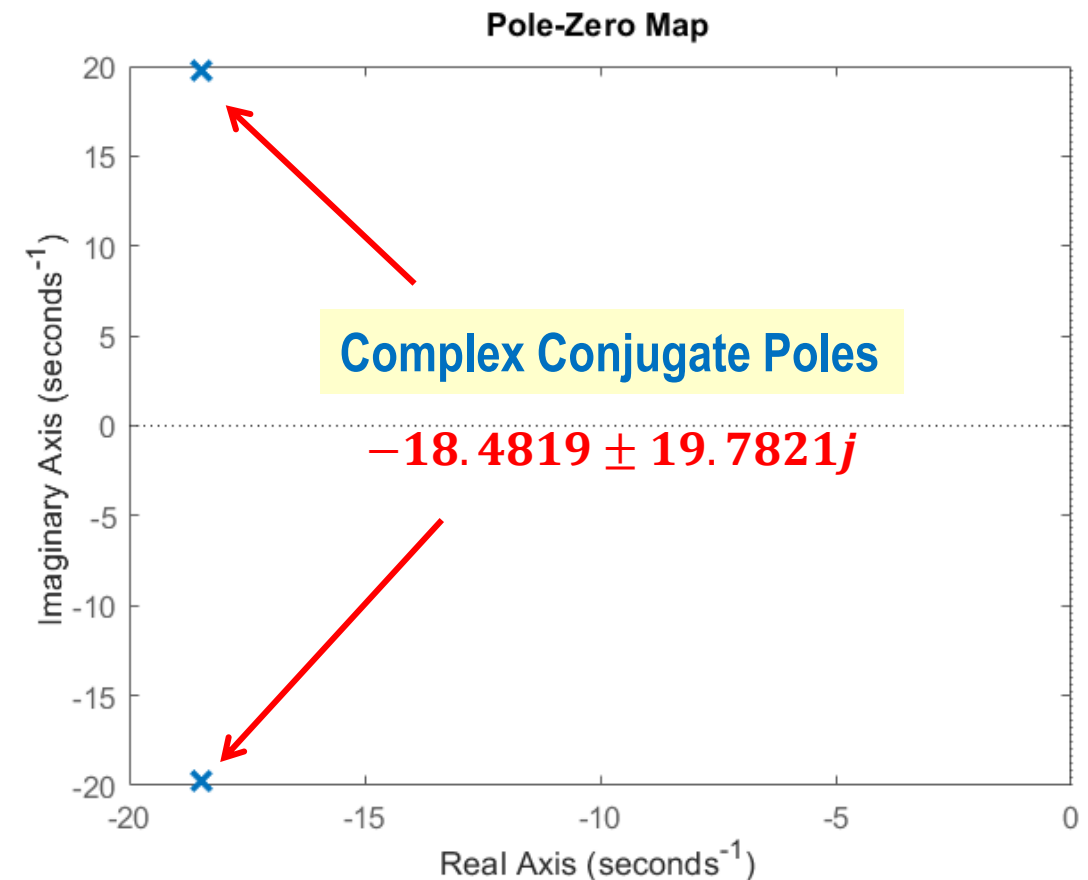
Noisy Data

Step 3: Select the parametric model structure, estimate and validate the model.

□ Second-order TF Model Validation:



□ Second-order TF Model Pole/Zero Map:



THANK YOU